IACR Transactions on Symmetric Cryptology
ISSN 2519-173X, Vol. 0, No. 0, pp. 0–0. DOI:10.46586/tosc.v0.i0.0-0

THF: Designing Low-Latency Tweakable Block Ciphers

Jianhua Wang[1,2,3], Tao Huang[3(✉)], Guang Zeng[3], Tianyou Ding[3,4,5], Shuang Wu[3] and Siwei Sun[1,2(✉)]

[1] School of Cryptology, University of Chinese Academy of Sciences, Beijing, China
wangjianhua@ucas.ac.cn,sunsiwei@ucas.ac.cn
[2] State Key Laboratory of Cryptology, P.O. Box 5159, Beijing, China
[3] Shield Lab, Huawei Technologies Co., Ltd. huangtao80@huawei.com,zengguang13@huawei.com,Wu.Shuang@huawei.com,dtykira@gmail.com
[4] State Key Laboratory of Cyberspace Security Defense, Institute of Information Engineering, CAS, Beijing, China
[5] School of Cyber Security, University of Chinese Academy of Sciences, Beijing, China

Abstract. We introduce the Three-Hash Framework (THF), a new instantiation of the LRW+ paradigm that employs three hash functions to process tweak inputs. We prove that THF achieves beyond-birthday-bound security under standard assumptions. By extending the general practical cryptanalysis framework to the multiple-tweak setting, we further demonstrate that THF offers balanced resistance to both single- and multiple-tweak attacks, thereby enabling the potential for lower latency compared to existing constructions. Building on this framework, we design `Blink`, a family of tweakable block ciphers optimized for ultra-low latency. `Blink` features logarithmic-depth Toeplitz-based hashing, which ensures efficient diffusion and scalability with varying tweak lengths. Our cryptanalysis shows that `Blink` achieves strong security with fewer rounds, while hardware evaluations confirm its superior latency performance. Notably, `Blink` maintains comparable latency even when the tweak length is doubled, underscoring the scalability advantage of THF.

Keywords: Low latency · THF · Hash function · Blink

1 Introduction

Symmetric-key cryptography plays a vital role in securing information across a wide array of applications, including communication protocols, financial systems, embedded devices, and cloud services. Over the years, the field has seen significant advancements, transitioning from general-purpose cryptographic designs to solutions tailored for specific domains. This evolution reflects the increasing demand for cryptographic primitives optimized for the unique requirements of various application scenarios.

Various memory protection solutions have been proposed to enhance the security of data stored in memory, such as Intel Software Guard Extensions (SGX) [Gue16], Intel Trust Domain Extensions (TDX) [Int20], AMD Secure Encrypted Virtualization (SEV) [AMD19], and ARM Confidential Compute Architecture (CCA) [ARM21]. In these schemes, a symmetric-key primitive is employed as the core cryptographic mechanism within the Memory Encryption Engine (MEE), which is implemented on the System on Chip (SoC) between the DRAM Controller and the system caches. The MEE plays a vital role in ensuring memory security by decrypting data when it is loaded from DRAM and verifying its integrity to detect potential tampering or corruption. Similarly, when data is written to the DRAM, the MEE encrypts the data and generates an integrity tag in

certain solutions such as TDX, which may be stored in the DRAM (either in the memory itself or by repurposing the ECC bits in DRAM).

However, the integration of the MEE in memory-intensive applications introduces additional overhead due to the encryption and decryption operations it performs. To maintain system performance and meet the real-time requirements of such applications, it is essential to minimize this latency. Therefore, the design of low-latency block ciphers becomes a key consideration in the development of memory protection solutions, as these ciphers must minimize the performance impact of the MEE while ensuring robust security.

Tweakable block ciphers [LRW02] enhance the flexibility of cryptographic algorithms by introducing a public tweak as an additional input, which allows each tweak to correspond to a different instantiation of the block cipher. This added flexibility significantly strengthens the adaptability of the algorithm for various applications. In the context of memory encryption, tweakable block ciphers can address specific challenges, such as the ciphertext repetition problem inherent in the ECB mode, by incorporating physical addresses or random nonces as tweaks.

There are two main approaches for designing tweakable block ciphers. One approach is based on modular methods, where tweakable block ciphers are constructed by extending traditional block cipher modes. Examples include LRW1 [LRW02], LRW2 [LRW02], and TNT [BGGS20] and so on [LST12a, JLM+19b, JN20, JKNS24]. The other approach is the TWEAKEY method [JNP14], which treats the tweak and key as a unified role. This method allows for the use of a single key schedule for both the key and tweak, streamlining the cipher's design. Many cryptographic instances have adopted the TWEAKEY method, including SKINNY [BJK+16], MANTIS [BJK+16], QARMA [Ava17] etc.

The design of low-latency cryptographic primitives is a relatively new and rapidly evolving research field, driven by the demand for efficient encryption in performance-critical applications. Among the earliest low-latency block ciphers is PRINCE [BCG+12], introduced in 2012 and specifically optimized for hardware latency. Building on its foundation, an updated version, PRINCEv2 [BEK+20], was proposed in 2020, offering enhanced security properties while maintaining its latency-optimized design.

Several other low-latency ciphers have followed. For example, MANTIS [BJK+16] provides a 64-bit block size, combining TWEAKEY principles with low-latency properties. Similarly, the QARMA family [Ava17] draws inspiration from both PRINCE and MANTIS, offering low-latency performance with support for 64-bit and 128-bit block sizes. Its updated variant, QARMAv2 [ABD+23], introduces enhancements such as support for larger tweaks and adaptations for applications like pointer authentication and memory encryption.

Other innovations in low-latency cipher design include SPEEDY [LMMR21], a family of block ciphers optimized specifically for latency. SPEEDY leverages a low-latency 6-bit S-box and employs a 192-bit block size. For cache randomization, SCARF [CGL+23] adopts an extremely compact 10-bit block size combined with a 240-bit key, achieving ultra-low latency. In the context of pointer authentication, BipBip [BDD+23] introduces a low-latency tweakable block cipher tailored to Intel's C^3 architecture. Notably, BipBip incorporates several innovative features such as a nonlinear tweak and key schedule, heterogeneous rounds, and a large master key, showcasing its adaptability to modern cryptographic requirements. Hu *et al.* proposed the uKNIT [HKPT24] design framework, which enables fine-grained, round-wise optimization of cryptographic primitives by allowing each round to employ distinct component structures. As a concrete instantiation of this framework, they designed the block cipher uKNIT-BC. Belkheyar *et al.* also introduced CHILOW [BDG+25], a family of tweakable block ciphers and a related pseudorandom function (PRF) specifically designed for embedded code encryption. At the core of CHILOW lies CHICHI, a novel family of nonlinear layers defined over even dimensions and constructed based on the well-known χ function.

In the realm of pseudo-random functions (PRFs), the low-latency design strategy of

`Orthros` [BIL+21], which uses branches of permutations to form a PRF, inspired the development of `Gleeok` [ABC+24]. This recent PRF supports a 256-bit key size, enhancing its applicability to cryptographic schemes requiring high security and efficiency. Furthermore, `Twinkle` [WHWL24] is a PRF designed with a large, tailored internal state, optimized to achieve low latency and minimal area overhead, making it highly suitable for authenticated encryption schemes in memory protection scenarios.

1.1 Motivation

Compared to a conventional block cipher, a tweakable block cipher includes an additional tweak input, which is public and typically adjustable. This feature allows an adversary to obtain plaintext-ciphertext pairs under multiple tweak values. Therefore, analyzing a cipher's resistance to multiple-tweak attacks is essential when evaluating tweakable block ciphers. Such attacks have been taken into account in the design of several tweakable low-latency ciphers, including `QARMA` [Ava17], `QARMAv2` [ABD+23], `Mantis` [BJK+16], `BipBip` [BDD+23], and `SCARF` [CGL+23], etc.

Existing low-latency tweakable block ciphers, such as `QARMA`, `QARMAv2`, and `MANTIS`, have achieved notable progress by optimizing round functions. However, we observe a significant disparity between their resistance to single-tweak and multiple-tweak differential attacks. For instance, in $\texttt{QARMA}_{64}$ with 16 rounds, the minimum number of active S-boxes in single-tweak trails is at least 74, but drops to only 48 in multiple-tweak trails. This gap mainly stems from the use of linear `TWEAKEY` schedules, which provide limited diffusion of tweak differences.

Although $\texttt{QARMAv2}_{64}$ refines the tweakey schedule of $\texttt{QARMA}_{64}$, its resistance still diminishes when the tweak length is doubled relative to the block size. Specifically, in 14 rounds, the minimum number of active S-boxes decreases from 47 (when the tweak length equals the block size) to 41 (when doubled). To preserve comparable security, $\texttt{QARMAv2}_{64}$ requires four additional rounds, thereby increasing latency.

These observations highlight a fundamental limitation of linear tweakey schedules: they reduce resistance against multiple-tweak attacks and hinder further latency optimization. This naturally motivate us to investigate whether redesigning or optimizing the tweak schedule can enhance resistance to multiple-tweak attacks, thereby enabling better latency characteristics. Ideally, such a tweak schedule should also exhibit strong scalability when the tweak length changes. That is, when instantiated in a concrete cipher, we hope that only minimal modifications are required, the corresponding security analyses can be extended seamlessly, and the overall latency increases only marginally.

1.2 Our Contributions

Building on the above motivation, we introduce a new mode for constructing tweakable block ciphers that significantly enhances resistance to multiple-tweak attacks while remaining scalable with respect to tweak length. Following the *prove-then-prune* methodology commonly adopted in modern symmetric cryptography (e.g., [MN17, HKR15, BGGS20]), we first establish the theoretical security of the mode and then instantiate a practical cipher within this framework. Our main contributions are summarized as follows:

Three-Hash Framework. We propose the Three-Hash Framework (THF), a new mode within the LRW+ paradigm [JKNS24], which integrates three pairwise-independent hash functions for processing tweaks. Given a plaintext $\boldsymbol{m}$ and a tweak $\boldsymbol{t}$, encryption is defined as

$$\boldsymbol{c} = \pi_4\Big(\pi_3\Big(\pi_2\big(\pi_1(\boldsymbol{m}) \oplus h_1(\boldsymbol{t})\big) \oplus h(\boldsymbol{t})\Big) \oplus h_2(\boldsymbol{t})\Big),$$

where $\pi_1, \pi_2, \pi_3, \pi_4$ are permutations and h_1, h, h_2 are hash functions.

We formally prove that THF achieves beyond-birthday-bound (BBB) security under standard assumptions. To assess its practical robustness, we extend the *general practical cryptanalysis* framework [FGGL+25] to the multiple-tweak setting. Compared with two representative BBB-secure LRW+ modes—4-LRW1 and 2-LRW2—THF demonstrates more balanced resistance to both single-tweak and multiple-tweak attacks. This improvement arises from its layered construction, which also enables partial parallelism: h_1 and π_1 can be computed concurrently during encryption, while h_2 and π_4^{-1} can be processed in parallel during decryption. In particular, the use of universal hash functions for tweak processing ensures scalability with respect to the tweak length, without compromising the overall security of the construction.

Design of a Low-Latency Tweakable Block Cipher. We instantiate THF with `Blink`, a tweakable block cipher family explicitly optimized for low latency. Its main design features include:

- *Logarithmic-depth hashing*: `Blink` adopts a hash function family based on Toeplitz matrices, which is inherently scalable with respect to tweak length and achieves circuit depth that grows only logarithmically with the tweak size. Compared to conventional linear tweak-processing methods, this hash construction significantly improves resistance against multiple-tweak differential and linear attacks, offering exponential security gains.

- *Latency-optimized round function*: The round function is inspired by `MIDORI`, using the same diffusion matrix but replacing the S-boxes with lower-latency alternatives and applying optimized cell permutations. This ensures high security while keeping the round function efficient.

- *Long master key*: Instead of implementing a conventional key schedule, `Blink` directly derives round keys by slicing a long master key. This approach simplifies encryption and decryption routines, reduces the need to repeatedly invoke a key schedule function, and enhances performance. Additionally, the longer key length improves the cipher's resistance against brute-force attacks and contributes to reduced latency.

We present a thorough cryptanalysis of `Blink`, particularly in the context of multiple-tweak attacks. Our results show that `Blink`, built atop THF, achieves robust security with fewer rounds than comparable designs. We further perform hardware evaluations that confirm `Blink`'s superior latency performance. These results validate the design principles behind THF, confirming its potential as a foundation for constructing ultra-low-latency tweakable block ciphers.

1.3 Organization

The remainder of this paper is organized as follows. Section 2 introduces the necessary preliminaries. Section 3 describes the THF framework and formally proves its security bounds. Section 4 explains the rationale for adopting the THF framework in designing low-latency tweakable block ciphers, supported by general practical cryptanalysis. In Section 5, we present the instantiation of THF through the cipher `Blink`. Section 6 elaborates on the design rationale and core principles behind `Blink`. A comprehensive security analysis is provided in Section 7. Section 8 evaluates the hardware performance of `Blink`. Finally, Section 9 concludes the paper.

2 Preliminaries

Let $\boldsymbol{x}$ represent a string and x_i the i-th bit of x. The set $\{0,1\}^n$ consists of all binary strings of length n, and $\mathsf{Perm}(n)$ denotes the set of all permutations over $\{0,1\}^n$. For any finite set $\mathcal{X}$, $|\mathcal{X}|$ represents its cardinality, and $x \leftarrow_{\$} \mathcal{X}$ indicates that x is sampled uniformly at random from $\mathcal{X}$.

2.1 Tweakable Block Cipher

A tweakable block cipher [LRW02] extends the functionality of a standard block cipher by introducing an additional input called the *tweak*. Unlike a traditional block cipher, which only takes a key and a plaintext to produce a ciphertext, a tweakable block cipher incorporates the tweak as a third input. Formally, it takes the key, tweak, and plaintext as inputs and outputs the ciphertext.

For a fixed value of the tweak, a tweakable block cipher can be viewed as a standard block cipher, where the tweak effectively modifies the encryption function without changing the key. This added flexibility allows tweakable block ciphers to support diverse cryptographic applications, such as disk encryption, where the tweak can represent sector or block indices to ensure ciphertext uniqueness even for an identical plaintext encrypted under the same key.

2.2 Some Special Function Families

A (τ, n)-hash function family $\mathcal{H}$ is defined as a set of functions $\{h : \{0,1\}^\tau \to \{0,1\}^n\}$. Moreover, a (τ, n)-hash function family $\mathcal{H}$ is called an *ϵ-almost XOR universal hash family (AXUHF)* if for all distinct $\boldsymbol{t}, \boldsymbol{t'} \in \{0,1\}^\tau$ and $\boldsymbol{\theta} \in \{0,1\}^n$,

$$\Pr\left(h \leftarrow_{\$} \mathcal{H} : h(\boldsymbol{t}) \oplus h(\boldsymbol{t'}) = \boldsymbol{\theta}\right) \leq \epsilon.$$

For the special case of $\boldsymbol{\theta} = 0^n$, $\mathcal{H}$ is referred to as an ϵ-AUHF.

A (τ, n)-tweakable permutation family $\widetilde{\mathcal{H}}$ is a family of functions $\{\widetilde{h} : \{0,1\}^\tau \times \{0,1\}^n \to \{0,1\}^n$ and $\forall \boldsymbol{t} \in \{0,1\}^\tau, \widetilde{h}(\boldsymbol{t}, \cdot) \in \mathsf{Perm}(n)\}$. $\widetilde{\mathcal{H}}$ is an *ϵ-almost universal tweakable permutation family (AUTPF)* if and only if for all distinct $(\boldsymbol{t}, \boldsymbol{m})$, $(\boldsymbol{t'}, \boldsymbol{m'}) \in \{0,1\}^\tau \times \{0,1\}^n$,

$$\Pr\left(\widetilde{h} \leftarrow_{\$} \widetilde{\mathcal{H}} : \widetilde{h}(\boldsymbol{t}, \boldsymbol{m}) = \widetilde{h}(\boldsymbol{t'}, \boldsymbol{m'})\right) \leq \epsilon.$$

2.3 Implementation of Boolean Functions

To mathematically model the latency of Boolean circuits for specific applications, we adopt the notion of *depth* as defined in [BBI+15].

Definition 1 (Depth)**.** The depth is defined as the sum of the sequential path delays of basic operations, namely `AND`, `OR`, `NAND`, `NOR`, and `NOT`.

Following the assumption in [BBI+15], we assign depth weights to basic logic operations as follows: `XOR/XNOR` are assigned a weight of 2, `AND/OR` a weight of 1.5, `NAND/NOR` a weight of 1, and `NOT` a weight of 0.5. This model serves as a practical initial guideline for estimating circuits' latency before conducting detailed hardware evaluations. The final latency figures will be obtained through evaluation using a standard cell library.[1]

[1] Although, as shown in [LMMR21], the actual latency of basic operations may vary depending on multiple factors, which can lead to deviations from the assigned depth weights, this measure remains highly valuable in cryptographic design. Many designs with small S-boxes based on this metric—such as those in `Midori` [BBI+15] and `Orthros` [BIL+21]—have demonstrated excellent hardware performance. Thus, we use these weights as initial design references, with final delay evaluations performed using a standard cell library.

2.4 Known Attacks on Symmetric Primitives

Differential Attacks. The differential probability between an input difference $\boldsymbol{\delta_{in}}$ and an output difference $\boldsymbol{\delta_{out}}$ for a function F is defined as

$$\Pr(\boldsymbol{\delta_{in}} \xrightarrow{F} \boldsymbol{\delta_{out}}) = \frac{|\{x \in \{0,1\}^n \mid F(x) \oplus F(x \oplus \boldsymbol{\delta_{in}}) = \boldsymbol{\delta_{out}}\}|}{2^n}.$$

A differential trail of a cipher $E_k = F_r \circ \cdots \circ F_1$ is represented by the sequence $(\boldsymbol{\delta_0}, \ldots, \boldsymbol{\delta_r})$, where each $\boldsymbol{\delta_i}$ denotes the intermediate difference after applying round function F_i. Assuming the independence of round keys, the probability of this trail is given by

$$\prod_{i=1}^{r} \Pr(\boldsymbol{\delta_{i-1}} \xrightarrow{F_i} \boldsymbol{\delta_i}).$$

The overall differential probability from input difference $\boldsymbol{\delta_0}$ to output difference $\boldsymbol{\delta_r}$ under the cipher E_k is then computed by summing over all possible intermediate trails:

$$\Pr(\boldsymbol{\delta_0} \xrightarrow{E_k} \boldsymbol{\delta_r}) = \sum_{(\boldsymbol{\delta_1}, \ldots, \boldsymbol{\delta_{r-1}})} \prod_{i=1}^{r} \Pr(\boldsymbol{\delta_{i-1}} \xrightarrow{F_i} \boldsymbol{\delta_i}).$$

Linear Attacks. The linear correlation between an input mask $\boldsymbol{\lambda_{in}}$ and an output mask $\boldsymbol{\lambda_{out}}$ of a function F is defined as

$$\mathrm{Cor}(\boldsymbol{\lambda_{in}} \xrightarrow{F} \boldsymbol{\lambda_{out}}) = 2\Pr\left({\boldsymbol{\lambda_{in}}}^T \cdot \boldsymbol{x} \oplus {\boldsymbol{\lambda_{out}}}^T \cdot F(\boldsymbol{x}) = 0\right) - 1,$$

where $\cdot$ denotes the matrix multiplication over $\mathbb{F}_2$.

A linear trail of a cipher $E_k = F_r \circ \cdots \circ F_1$ is described by the sequence $(\boldsymbol{\lambda_0}, \boldsymbol{\lambda_1}, \ldots, \boldsymbol{\lambda_r})$, where each $\boldsymbol{\lambda_i}$ denotes the linear mask after round function F_i. Assuming the independence of round keys, the correlation of the trail is given by

$$\prod_{i=1}^{r} \mathrm{Cor}(\boldsymbol{\lambda_{i-1}} \xrightarrow{F_i} \boldsymbol{\lambda_i}).$$

The overall linear correlation from input mask $\boldsymbol{\lambda_0}$ to output mask $\boldsymbol{\lambda_r}$ under the cipher E_k is computed by summing over all possible intermediate trails:

$$\mathrm{Cor}(\boldsymbol{\lambda_0} \xrightarrow{E_k} \boldsymbol{\lambda_r}) = \sum_{\boldsymbol{\lambda_1}, \ldots, \boldsymbol{\lambda_{r-1}}} \prod_{i=1}^{r} \mathrm{Cor}(\boldsymbol{\lambda_{i-1}} \xrightarrow{F_i} \boldsymbol{\lambda_i}).$$

Algebraic Attacks. An n-variable Boolean function f can be uniquely expressed in its algebraic normal form (ANF) as

$$f(\boldsymbol{x}) = \bigoplus_{\boldsymbol{v} \in \mathbb{F}_2^n} a_{\boldsymbol{v}} \boldsymbol{x}^{\boldsymbol{v}}, \quad a_{\boldsymbol{v}} \in \mathbb{F}_2,$$

where $\boldsymbol{x}^{\boldsymbol{v}} = x_1^{v_1} x_2^{v_2} \cdots x_n^{v_n}$.

Given a fixed $\boldsymbol{u} \in \mathbb{F}_2^n$, the function $f(\boldsymbol{x})$ can be rewritten as

$$f(\boldsymbol{x}) = \sum_{\boldsymbol{v} \preceq \boldsymbol{u}} g_{\boldsymbol{u},\boldsymbol{v}}(\boldsymbol{x}) \cdot \boldsymbol{x}^{\boldsymbol{v}},$$

where

$$g_{\boldsymbol{u},\boldsymbol{v}}(\boldsymbol{x}) = \bigoplus_{\boldsymbol{w} \preceq \boldsymbol{u}^c} a_{\boldsymbol{w} \oplus \boldsymbol{v}} \boldsymbol{x}^{\boldsymbol{w}}.$$

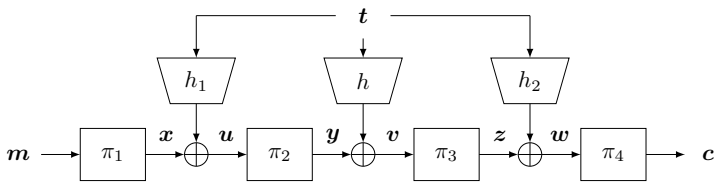

Figure 1: The diagram of THF mode

Here, $\boldsymbol{u}^c$ denotes the bitwise complement of $\boldsymbol{u}$, i.e., the i-th bit of $\boldsymbol{u}^c$ is $u_i \oplus 1$. The partial order $\boldsymbol{v} \preceq \boldsymbol{u}$ indicates that $v_i \leq u_i$ for all i.

The *algebraic degree* of f with respect to the variable subset $\boldsymbol{x}_{\boldsymbol{u}}$ (i.e., those x_i such that $u_i = 1$) is defined as

$$\deg(f, \boldsymbol{u}) = \max_{\boldsymbol{v} \preceq \boldsymbol{u}} \{\mathrm{wt}(\boldsymbol{v}) \mid g_{\boldsymbol{u},\boldsymbol{v}}(\boldsymbol{x}) \not\equiv 0\},$$

where $\mathrm{wt}(\boldsymbol{v})$ denotes the Hamming weight of $\boldsymbol{v}$.

The algebraic degree serves as a key indicator of a Boolean function's resistance to algebraic attacks: higher degree typically implies stronger security.

3 A New Scheme for Tweak Scheduling

We propose a new instantiation of the LRW+ mode [JKNS24], called the Three-Hash Framework (THF), which employs three hash functions to process the tweak, thereby supporting tweaks of arbitrary length. Unlike the TWEAKEY framework that treats the key and tweak uniformly, THF deliberately separates their roles, as related-key attacks are outside our threat model. Instead, the design focuses on strengthening resistance to multiple-tweak attacks. In the following, we detail the construction of THF and provide a formal theoretical security analysis.

3.1 The THF Mode

Let $\mathcal{H}$ and $\mathcal{H}'$ be two (τ, n)-hash function families. For $h \in \mathcal{H}$, $h_1, h_2 \in \mathcal{H}'$, and permutations $\pi_i \in \mathsf{Perm}(n)$ for $i \in \{1, \ldots, 4\}$, the encryption of a plaintext $\boldsymbol{m} \in \{0,1\}^n$ under a tweak $\boldsymbol{t} \in \{0,1\}^\tau$ proceeds as follows (illustrated in Figure 1):

$$\boldsymbol{c} = \pi_4\left(\pi_3\left(\pi_2\left(\pi_1(\boldsymbol{m}) \oplus h_1(\boldsymbol{t})\right) \oplus h(\boldsymbol{t})\right) \oplus h_2(\boldsymbol{t})\right),$$

where $\boldsymbol{c} \in \{0,1\}^n$ is the resulting ciphertext.

3.2 Security of the THF

THF is an instance of the LRW+ construction [JKNS24], which is a (τ, n)-tweakable permutation family introduced by Jha *et al.* at EUROCRYPT 2024. Let $\widetilde{\mathcal{H}}$ represent a family of (τ, n)-tweakable permutations. The LRW+ construction is defined by the following mapping:

$$\boldsymbol{c} = \widetilde{h_2}^{-1}\left(\boldsymbol{t}, \pi_2\left(h(\boldsymbol{t}) \oplus \pi_1\left(\widetilde{h_1}(\boldsymbol{t}, \boldsymbol{m})\right)\right)\right)$$

where $\widetilde{h_1}, \widetilde{h_2} \leftarrow_\$ \widetilde{\mathcal{H}}$, $\pi_1, \pi_2 \rightarrow_\$ \mathsf{Perm}(n)$, $h \leftarrow_\$ \mathcal{H}$ and for all $\boldsymbol{t}$, $\widetilde{h_2}^{-1}(\boldsymbol{t}, \cdot)$ represents the inverse of $\widetilde{h_2}(\boldsymbol{t}, \cdot)$. The security of LRW+ relies on the properties of $\widetilde{\mathcal{H}}$ and $\mathcal{H}$, as outlined in Theorem 1. For more details, please refer to [JKNS24].

Theorem 1 ([JKNS24]). *Let $0 \leq \epsilon_1, \epsilon_2 \leq 1$. Suppose $\widetilde{\mathcal{H}}$ is an ϵ_1-AUTPF, and $\mathcal{H}$ is an ϵ_2-AUHF. Further assume $\widetilde{h_1}, \widetilde{h_2}$ and h are pairwise independent. Then for any $q \leq 2^{n-2}$ the IND-CCA advantage of LRW+ is bounded by*

$$\mathbf{Adv}^{\text{ind-cca}}_{\mathit{LRW+}}(q) \leq \epsilon(q, n),$$

where

$$\begin{aligned}\epsilon(q,n) = 2q^2\epsilon_1^{1.5} + 2^{2-n}q^4\epsilon_1^2 + 2^{5-2n}q^4\epsilon_1 + 13 \cdot 2^{-3n}q^4 \\ + q^2\epsilon_1^2 + q^2\epsilon_1\epsilon_2 + 2^{1-2n}q^2.\end{aligned}$$

In the THF construction (see Figure 1), the mappings $\pi_1(\boldsymbol{m}) \oplus h_1(\boldsymbol{t})$ and $\pi_4\left(h_2(\boldsymbol{t}) \oplus \boldsymbol{z}\right)$ correspond to the (τ, n)-tweakable permutations $\widetilde{h_1}$ and $\widetilde{h_2}^{-1}$, respectively, in the LRW+ paradigm. Similarly, π_2, π_3, and h play the roles of the intermediate permutations and the central hash function, respectively.

Notably, we have $\left(\pi_4\left(h_2(\boldsymbol{t}) \oplus \boldsymbol{z}\right)\right)^{-1} = {\pi_4}^{-1}(\boldsymbol{c}) \oplus h_2(\boldsymbol{t})$. Therefore, to analyze the security bounds of THF via Theorem 1, it is crucial to study the properties of the tweakable permutation family

$$\widetilde{\mathcal{H}'} := \{\pi(\boldsymbol{m}) \oplus h(\boldsymbol{t}) \mid \pi \in \mathcal{P},\, h \in \mathcal{H}'\},$$

where $\mathcal{P} \subseteq \mathsf{Perm}(n)$.

Proposition 1. *If $\widetilde{\mathcal{H}'}$ is an ϵ-AUTPF, then $\mathcal{H}'$ must be an ϵ-AUHF.*

Proof. By the definition of ϵ-AUTPF, we know that for any distinct pairs $(\boldsymbol{m}, \boldsymbol{t})$ and $(\boldsymbol{m'}, \boldsymbol{t'})$, it holds that

$$\begin{aligned}&\Pr\left(\widetilde{h} \leftarrow_{\$} \widetilde{\mathcal{H}'} : \widetilde{h}(\boldsymbol{t}, \boldsymbol{m}) = \widetilde{h}(\boldsymbol{t'}, \boldsymbol{m'})\right) \leq \epsilon \\ \Leftrightarrow &\Pr\left(\pi \leftarrow_{\$} \mathcal{P}, h \leftarrow_{\$} \mathcal{H}' : \pi(\boldsymbol{m}) \oplus h(\boldsymbol{t}) = \pi(\boldsymbol{m'}) \oplus h(\boldsymbol{t'})\right) \leq \epsilon.\end{aligned}$$

If we fix $\boldsymbol{m} = \boldsymbol{m'}$, this simplifies to:

$$\Pr\left(h \leftarrow_{\$} \mathcal{H}' : h(\boldsymbol{t}) = h(\boldsymbol{t'})\right) \leq \epsilon,$$

which implies that $\mathcal{H}'$ is an ϵ-AUHF. □

Proposition 1 indicates that the condition of $\mathcal{H}'$ being an ϵ-AUHF is necessary for $\widetilde{\mathcal{H}'}$ to be an ϵ-AUTPF. Furthermore, we observed that a stronger property of $\mathcal{H}'$ suffices to ensure that $\widetilde{\mathcal{H}'}$ is indeed an ϵ-AUTPF, as stated in Lemma 1.

Lemma 1. *If $\mathcal{H}'$ is an ϵ-AXUHF, then for any non-empty $\mathcal{P} \subseteq \mathsf{Perm}(n)$, $\widetilde{\mathcal{H}'}$ must be an ϵ-AUTPF.*

Proof. For any distinct pairs $(\boldsymbol{m}, \boldsymbol{t})$ and $(\boldsymbol{m'}, \boldsymbol{t'})$, we consider two cases: (1) $\boldsymbol{t} = \boldsymbol{t'}$ and (2) $\boldsymbol{t} \neq \boldsymbol{t'}$.

- **Case 1**: If $\boldsymbol{t} = \boldsymbol{t'}$, then $\boldsymbol{m} \neq \boldsymbol{m'}$. Since π is a permutation, it follows that $\pi(\boldsymbol{m}) \neq \pi(\boldsymbol{m'})$. Then,

$$\begin{aligned}&\Pr\left(\widetilde{h} \leftarrow_{\$} \widetilde{\mathcal{H}'} : \widetilde{h}(\boldsymbol{t}, \boldsymbol{m}) = \widetilde{h}(\boldsymbol{t'}, \boldsymbol{m'})\right) \\ =&\Pr\left(\pi \leftarrow_{\$} \mathcal{P}, h \leftarrow_{\$} \mathcal{H}' : \pi(\boldsymbol{m}) \oplus h(\boldsymbol{t}) = \pi(\boldsymbol{m'}) \oplus h(\boldsymbol{t'})\right) \\ =&\Pr\left(\pi \leftarrow_{\$} \mathcal{P} : \pi(\boldsymbol{m}) = \pi(\boldsymbol{m'})\right) \\ =&0.\end{aligned}$$

- **Case 2**: If $\boldsymbol{t} \neq \boldsymbol{t'}$, we proceed as follows:

$$\Pr\left(\widetilde{h} \leftarrow_{\$} \widetilde{\mathcal{H}'} : \widetilde{h}(\boldsymbol{t}, \boldsymbol{m}) = \widetilde{h}(\boldsymbol{t'}, \boldsymbol{m'})\right) \tag{1}$$

$$= \Pr\left(\pi \leftarrow_{\$} \mathcal{P}, h \leftarrow_{\$} \mathcal{H}' : \pi(\boldsymbol{m}) \oplus h(\boldsymbol{t}) = \pi(\boldsymbol{m'}) \oplus h(\boldsymbol{t'})\right) \tag{2}$$

$$= \sum_{\boldsymbol{\theta} \in \{0,1\}^n} \Pr\left(\pi \leftarrow_{\$} \mathcal{P} : \pi(\boldsymbol{m}) \oplus \pi(\boldsymbol{m'}) = \boldsymbol{\theta}\right) \cdot \Pr\left(h \leftarrow_{\$} \mathcal{H}' : h(\boldsymbol{t}) \oplus h(\boldsymbol{t'}) = \boldsymbol{\theta}\right) \tag{3}$$

$$\leq \epsilon \cdot \sum_{\boldsymbol{\theta} \in \{0,1\}^n} \Pr\left(\pi \leftarrow_{\$} \mathcal{P} : \pi(\boldsymbol{m}) \oplus \pi(\boldsymbol{m'}) = \boldsymbol{\theta}\right) \tag{4}$$

$$= \epsilon. \tag{5}$$

  The inequation (4) follows from the fact that $\mathcal{H}'$ is an ϵ-AXUHF.

Therefore, $\widetilde{\mathcal{H}'}$ meets the requirements of an ϵ-AUTPF. □

Therefore, the security bound of the THF mode can be directly derived, which is stated in Theorem 2. If $\epsilon_1, \epsilon_2 = \mathcal{O}(2^{-n})$, then the CCA security bound of the THF is up to $\mathcal{O}(2^{3/4n})$.

Theorem 2. *Let* $0 \leq \epsilon_1, \epsilon_2 \leq 1$, $h_1, h_2 \leftarrow_{\$} \mathcal{H}'$, $h \leftarrow_{\$} \mathcal{H}$ *and* $\pi_2, \pi_3 \leftarrow_{\$} \mathsf{Perm}(n)$. *If* $\mathcal{H}'$ *is an* ϵ_1*-AXUHF, and* $\mathcal{H}$ *is an* ϵ_2*-AUHF. Besides,* $h_1(\boldsymbol{t}) \oplus \pi_1(\boldsymbol{m})$, $h_2(\boldsymbol{t}) \oplus \pi_4^{-1}(\boldsymbol{m})$ *and* h *are pairwise independent. Then, for* $q \leq 2^{n-2}$, *we have*

$$\mathbf{Adv}^{ind-cca}_{THF}(q) \leq \epsilon(q, n),$$

where $\epsilon(q,n) = 2q^2\epsilon_1^{1.5} + 2^{2-n}q^4\epsilon_1^2 + 2^{5-2n}q^4\epsilon_1 + 13 \cdot 2^{-3n}q^4 + q^2\epsilon_1^2 + q^2\epsilon_1\epsilon_2 + 2^{1-2n}q^2$.

Related-cipher Attacks. Different tweak lengths correspond to different cipher instances. For different tweak input lengths, some hash functions may have trivial collisions (see [SSW23]). These collisions could lead to vulnerabilities between different cipher instances. To mitigate potential security risks, one can incorporate the tweak length into the design of the hash function. However, in this paper, we do not make any claims regarding the attack security between these different cipher instances.

4 Rationale for THF Mode

In practice, symmetric cipher designers often follow a *"prove-then-prune"* methodology: they begin with constructions that offer strong provable security guarantees and then incrementally optimize for performance, provided that the essential security properties remain intact.

In this section, we assess the practical security of tweakable block cipher modes under the *general practical cryptanalysis* framework [FGGL+25], with a particular focus on resistance to multiple-tweak attacks.

We focus our analysis on three constructions: 4-LRW1, 2-LRW2, and THF. The first two are well-studied and widely regarded as the minimal-permutation instantiations of the CLRW1 and CLRW2 paradigms, respectively, that still achieve BBB CCA-security in the idealized model. Moreover, both share structural affinities with the THF design, making them natural benchmarks for comparison.

Specifically, the THF mode inherits the structural blueprint of 4-LRW1 but replaces identity-based tweak processing with hash-based mechanisms. Additionally, the first and fourth permutations in THF are not required to be random. Compared to 2-LRW2, THF introduces precisely two additional outer permutations corresponding to the aforementioned first and fourth permutations.

Our evaluation will enable an intuitive and fair comparison of the security margins attainable when constructing tweakable block ciphers from standard block ciphers via these modes. In addition, we will provide a latency analysis under comparable security assumptions, highlighting the THF mode's greater potential for low-latency tweakable block cipher design.

Settings. Let $\boldsymbol{m} \in \{0,1\}^n$ and $\boldsymbol{t} \in \{0,1\}^\tau$ be the message and tweak, respectively. The encryption functions of the three constructions are defined as follows:

$$E_{\mathsf{THF}} = R^d\Big(R^c\Big(R^b\big(R^a(\boldsymbol{m}) \oplus h_1(\boldsymbol{t})\big) \oplus h(\boldsymbol{t})\Big) \oplus h_2(\boldsymbol{t})\Big),$$

$$E_{\text{4-LRW1}} = R^{\tilde{d}}\Big(R^{\tilde{c}}\Big(R^{\tilde{b}}\big(R^{\tilde{a}}(\boldsymbol{m}) \oplus \boldsymbol{t}\big) \oplus \boldsymbol{t}\Big) \oplus \boldsymbol{t}\Big),$$

$$E_{\text{2-LRW2}} = R^{\hat{c}}\Big(R^{\hat{b}}\big(\boldsymbol{m} \oplus h_1(\boldsymbol{t})\big) \oplus h(\boldsymbol{t})\Big) \oplus h_2(\boldsymbol{t}),$$

where $R^i(\cdot)$ denotes the application of i rounds of a keyed round function R, $\tau = n$ for $E_{\text{4-LRW1}}$, and $h = h_1 \oplus h_2$. Moreover, all parameters $b, \ldots, \hat{c}$, except for a and d, are strictly positive, and $a, d \geq 0$.

We assume that h_1 and h_2 are independently drawn from a 2^{-n}-AXUHF, with equal latency. The block size is assumed to satisfy $n \geq 64$, as required by our design.

The Latency Evaluation. Under this assumption, and by neglecting the latency introduced by a small number of XOR operations, the total latency (denoted by $T(\cdot)$) of each construction can be estimated as follows:

$$T\big(E_{\mathsf{THF}}\big) = \max\big(T(h_1),\ a \cdot T(R)\big) + (b + c + d) \cdot T\big(R\big),$$

$$T\big(E_{\text{4-LRW1}}\big) = (\tilde{a} + \tilde{b} + \tilde{c} + \tilde{d}) \cdot T\big(R\big),$$

$$T\big(E_{\text{2-LRW2}}\big) = T(h_1) + (\hat{b} + \hat{c}) \cdot T\big(R\big).$$

Remark. In the following analysis, since not all expressions allow for direct quantitative comparison, we adopt a design-oriented perspective to derive general judgments. Specifically, we intentionally disregard highly exceptional or degenerate cases and focus on typical, broadly applicable conclusions. For instance, we may assume that instances with more rounds generally offer stronger security, without delving into edge cases where specific properties of the round function may lead to unexpected degradation.

4.1 The Security of Constructions

To maintain the security of the constructions, note that certain components are required to behave as random permutations. Consequently, their instantiations R^i must satisfy a minimum number of rounds to prevent structural collision-based attacks. Since we will analyze major practical attacks in later sections, we do not assume that these R^i's behave as full random permutations. Instead, we adopt an aggressive (pruning-based) approach and aim to relax the rounds requirements as much as possible, subject to the constraint that the adversary cannot mount a construction-based collision attack by recovering some R^i through key guessing within the security bounds. Let us denote the minimal number of rounds required to ensure this condition by r_C. Then we assume:

$$b, c, \tilde{a}, \tilde{b}, \tilde{c}, \tilde{d}, \hat{b}, \hat{c} \geq r_C.$$

4.2 Single-tweak attacks

The single-tweak (ST) security of each construction reduces to the security of the underlying round functions:

$$E_{\mathsf{THF}},\ E_{\mathsf{4\text{-}LRW1}},\ E_{\mathsf{2\text{-}LRW2}} \iff R^{a+b+c+d},\ R^{\tilde{a}+\tilde{b}+\tilde{c}+\tilde{d}},\ R^{\hat{b}+\hat{c}},\ \text{respectively.}$$

Assuming that the round function R requires at least r_S rounds to meet the desired security level, we obtain the following lower bounds:

$$a+b+c+d \geq r_S, \quad \tilde{a}+\tilde{b}+\tilde{c}+\tilde{d} \geq r_S, \quad \hat{b}+\hat{c} \geq r_S.$$

4.3 Multiple-tweak attacks

We mainly consider three highly threatening categories of multiple-tweak (MT) attacks: MT differential attacks, MT linear attacks, and MT algebraic attacks.

4.3.1 MT Differential attacks

Given differences $\boldsymbol{\delta_m}, \boldsymbol{\delta_t}, \boldsymbol{\delta_c} \in \{0,1\}^n$ with at least two of them nonzero, we analyze the differential propagation

$$(\boldsymbol{\delta_m}, \boldsymbol{\delta_t}) \xrightarrow{E} \boldsymbol{\delta_c}.$$

- For $E = E_{\mathsf{THF}}$, all trails follow a unified structure determined by the choice of $\boldsymbol{\delta_1}, \boldsymbol{\delta_2}, \boldsymbol{\delta_3}, \boldsymbol{\gamma_1}, \boldsymbol{\gamma_2}$, and the probability of each trail is given by

$$\begin{aligned} p_{\mathsf{THF}} = \ & \Pr(\boldsymbol{\delta_m} \xrightarrow{R^a} \boldsymbol{\delta_1}) \cdot \Pr(\boldsymbol{\delta_1} \oplus \boldsymbol{\gamma_1} \xrightarrow{R^b} \boldsymbol{\delta_2}) \cdot \\ & \Pr(\boldsymbol{\delta_2} \oplus \boldsymbol{\gamma_1} \oplus \boldsymbol{\gamma_2} \xrightarrow{R^c} \boldsymbol{\delta_3}) \cdot \Pr(\boldsymbol{\delta_3} \oplus \boldsymbol{\gamma_2} \xrightarrow{R^d} \boldsymbol{\delta_c}) \cdot \\ & \Pr(\boldsymbol{\delta_t} \xrightarrow{h_1} \boldsymbol{\gamma_1}) \cdot \Pr(\boldsymbol{\delta_t} \xrightarrow{h_2} \boldsymbol{\gamma_2}). \end{aligned}$$

  We first consider the trivial case where $\boldsymbol{\delta_t} = \mathbf{0}$. In this case, by setting $\boldsymbol{\gamma_1} = \boldsymbol{\gamma_2} = \mathbf{0}$, the probability p_{THF} coincides with that in the ST setting.

  We now focus on the non-trivial case where $\boldsymbol{\delta_t} \neq \mathbf{0}$. Since h_1 and h_2 are independently sampled from a 2^{-n}-AXUHF, for any $\boldsymbol{\gamma_1}, \boldsymbol{\gamma_2} \in \{0,1\}^n$, it holds that

$$\Pr(\boldsymbol{\delta_t} \xrightarrow{h_1} \boldsymbol{\gamma_1}) = \Pr(\boldsymbol{\delta_t} \xrightarrow{h_2} \boldsymbol{\gamma_2}) = 2^{-n}.$$

  Consequently, the differential probability p_{THF} is upper bounded by 2^{-2n}. Furthermore, when $n \geq 64$, it becomes practically infeasible to construct more than 2^n valid differential trails of the form

$$(\boldsymbol{\delta_m}, \boldsymbol{\delta_t}) \xrightarrow{E} \boldsymbol{\delta_c}$$

  such that their cumulative probability exceeds 2^{-n}, which is typically required for a successful differential attack.

  Therefore, we conclude that when the block size satisfies $n \geq 64$, E_{THF} is secure against the MT differential analysis.

- For $E = E_{\mathsf{4\text{-}LRW1}}$, all trails are characterized by the sequence $\boldsymbol{\delta_1}, \boldsymbol{\delta_2}, \boldsymbol{\delta_3}$, and the probability of each trail is

$$\begin{aligned} p_{\mathsf{4\text{-}LRW1}} = & \Pr(\boldsymbol{\delta_m} \xrightarrow{R^{\tilde{a}}} \boldsymbol{\delta_1}) \cdot \Pr(\boldsymbol{\delta_1} \oplus \boldsymbol{\delta_t} \xrightarrow{R^{\tilde{b}}} \boldsymbol{\delta_2}) \cdot \\ & \Pr(\boldsymbol{\delta_2} \oplus \boldsymbol{\delta_t} \xrightarrow{R^{\tilde{c}}} \boldsymbol{\delta_3}) \cdot \Pr(\boldsymbol{\delta_3} \oplus \boldsymbol{\delta_t} \xrightarrow{R^{\tilde{d}}} \boldsymbol{\delta_c}). \end{aligned}$$

Here, the tweak difference $\boldsymbol{\delta_t}$ is actively involved in each component of the cipher and can be deliberately chosen to maximize the transition probability. Consequently, for $\boldsymbol{\delta_m}, \boldsymbol{\delta_c} \neq \mathbf{0}$, the maximum differential probability over all choices of $\boldsymbol{\delta_t}$ satisfies

$$\max_{\boldsymbol{\delta_t}} p_{\text{4-LRW1}} \geq \Pr(\boldsymbol{\delta_m} \xrightarrow{R^{\tilde{a}}} \boldsymbol{\delta_1}) \cdot \Pr(\boldsymbol{\delta_1} \xrightarrow{R^{\tilde{b}}} \boldsymbol{\delta_2}) \cdot \Pr(\boldsymbol{\delta_2} \xrightarrow{R^{\tilde{c}}} \boldsymbol{\delta_3}) \cdot \Pr(\boldsymbol{\delta_3} \xrightarrow{R^{\tilde{d}}} \boldsymbol{\delta_c}).$$

This indicates that, in the MT setting, the maximum probability of differential trails is always at least as large as in the ST case. In addition, the following two cases can also potentially lead to differential trails with higher probability. For the case $\boldsymbol{\delta_m} = \mathbf{0}$, by setting $\boldsymbol{\delta_1} = \mathbf{0}$, the trail probability simplifies to

$$p_{\text{4-LRW1}} = \Pr(\boldsymbol{\delta_t} \xrightarrow{R^{\tilde{b}}} \boldsymbol{\delta_2}) \cdot \Pr(\boldsymbol{\delta_2} \oplus \boldsymbol{\delta_t} \xrightarrow{R^{\tilde{c}}} \boldsymbol{\delta_3}) \cdot \Pr(\boldsymbol{\delta_3} \oplus \boldsymbol{\delta_t} \xrightarrow{R^{\tilde{d}}} \boldsymbol{\delta_c}).$$

Similarly, when $\boldsymbol{\delta_c} = \mathbf{0}$, setting $\boldsymbol{\delta_3} = \boldsymbol{\delta_t}$ yields

$$p_{\text{4-LRW1}} = \Pr(\boldsymbol{\delta_m} \xrightarrow{R^{\tilde{a}}} \boldsymbol{\delta_1}) \cdot \Pr(\boldsymbol{\delta_1} \oplus \boldsymbol{\delta_t} \xrightarrow{R^{\tilde{b}}} \boldsymbol{\delta_2}) \cdot \Pr(\boldsymbol{\delta_2} \oplus \boldsymbol{\delta_t} \xrightarrow{R^{\tilde{c}}} \boldsymbol{\delta_t}).$$

These observations suggest that the resistance of $E_{\text{4-LRW1}}$ against MT differential attacks is typically weaker than that against ST differential attacks.

- For $E = E_{\text{2-LRW2}}$, the differential trails are defined by $\boldsymbol{\delta_1}, \boldsymbol{\delta_2}, \boldsymbol{\gamma_1}, \boldsymbol{\gamma_2}$, and the probability of each trail is

$$\begin{aligned} p_{\text{2-LRW2}} = &\Pr(\boldsymbol{\delta_m} \oplus \boldsymbol{\gamma_1} \xrightarrow{R^{\tilde{b}}} \boldsymbol{\delta_2}) \cdot \Pr(\boldsymbol{\delta_2} \oplus \boldsymbol{\gamma_1} \oplus \boldsymbol{\gamma_2} \xrightarrow{R^{\tilde{c}}} \boldsymbol{\delta_c} \oplus \boldsymbol{\gamma_2}) \cdot \\ &\Pr(\boldsymbol{\delta_t} \xrightarrow{h_1} \boldsymbol{\gamma_1}) \cdot \Pr(\boldsymbol{\delta_t} \xrightarrow{h_2} \boldsymbol{\gamma_2}). \end{aligned}$$

Similarly to the analysis of E_{THF}, we conclude that $E_{\text{2-LRW2}}$ achieves resistance against MT differential attacks, provided that $n \geq 64$.

4.3.2 MT Linear Attacks

Given masks $\boldsymbol{\lambda_m}, \boldsymbol{\lambda_t}, \boldsymbol{\lambda_c} \in \{0,1\}^n$ with at least two of them nonzero, we consider the linear propagation:

$$(\boldsymbol{\lambda_m}, \boldsymbol{\lambda_t}) \xrightarrow{E} \boldsymbol{\lambda_c}.$$

- For $E = E_{\mathsf{THF}}$, all linear trails can be characterized by a fixed selection of intermediate masks $\boldsymbol{\lambda_1}, \boldsymbol{\lambda_2}, \boldsymbol{\lambda_3}$, and the correlation of each trail is given by:

$$\begin{aligned} c_{\mathsf{THF}} = &\operatorname{Cor}(\boldsymbol{\lambda_m} \xrightarrow{R^a} \boldsymbol{\lambda_1}) \cdot \operatorname{Cor}(\boldsymbol{\lambda_1} \xrightarrow{R^b} \boldsymbol{\lambda_2}) \cdot \operatorname{Cor}(\boldsymbol{\lambda_2} \xrightarrow{R^c} \boldsymbol{\lambda_3}) \\ &\cdot \operatorname{Cor}(\boldsymbol{\lambda_3} \xrightarrow{R^d} \boldsymbol{\lambda_c}) \cdot \operatorname{Cor}\left(\boldsymbol{\lambda_t} \xrightarrow{h_1, h_2, h_1 \oplus h_2} \boldsymbol{\lambda_1}, \boldsymbol{\lambda_3}, \boldsymbol{\lambda_2}\right). \end{aligned} \tag{6}$$

We define the tweak-related correlation term as

$$\operatorname{Cor}\left(\boldsymbol{\lambda_t} \xrightarrow{h_1, h_2, h_1 \oplus h_2} \boldsymbol{\lambda_1}, \boldsymbol{\lambda_3}, \boldsymbol{\lambda_2}\right) \triangleq c_h, \tag{7}$$

which satisfies, by definition,

$$c_h = 2 \Pr\left(\boldsymbol{\lambda_t}^T \cdot \boldsymbol{t} \oplus (\boldsymbol{\lambda_1} \oplus \boldsymbol{\lambda_2})^T \cdot h_1 \oplus (\boldsymbol{\lambda_3} \oplus \boldsymbol{\lambda_2})^T \cdot h_2 = 0\right) - 1.$$

If the tweak $\boldsymbol{t}$ is fixed, then $c_h = 1$, which is equivalent to the ST scenario.

When the tweak varies, and both plaintext and ciphertext are also allowed to vary, the overall trail correlation c_{THF} cannot exceed that in the ST case. In particular, if either the plaintext or ciphertext is fixed, the correlation simplifies to:

$$c_{\mathsf{THF}} = \mathrm{Cor}(\boldsymbol{\lambda_1} \xrightarrow{R^b} \boldsymbol{\lambda_2}) \cdot \mathrm{Cor}(\boldsymbol{\lambda_2} \xrightarrow{R^c} \boldsymbol{\lambda_3}) \cdot \mathrm{Cor}(\boldsymbol{\lambda_3} \xrightarrow{R^d} \boldsymbol{\lambda_c}) \cdot c_h,$$

or

$$c_{\mathsf{THF}} = \mathrm{Cor}(\boldsymbol{\lambda_m} \xrightarrow{R^a} \boldsymbol{\lambda_1}) \cdot \mathrm{Cor}(\boldsymbol{\lambda_1} \xrightarrow{R^b} \boldsymbol{\lambda_2}) \cdot \mathrm{Cor}(\boldsymbol{\lambda_2} \xrightarrow{R^c} \boldsymbol{\lambda_3}) \cdot c_h.$$

These observations highlight that the linear security of THF in the MT setting also depends on the value of c_h. A smaller c_h leads to a lower overall correlation, thereby enhancing the cipher's resistance to MT linear attacks.

- For $E = E_{\text{4-LRW1}}$, all trails are characterized by the choice of $\boldsymbol{\lambda_1}, \boldsymbol{\lambda_2}$, and the trail correlation is:

$$\begin{aligned} c_{\text{4-LRW1}} = \mathrm{Cor}(\boldsymbol{\lambda_m} \xrightarrow{R^{\tilde{a}}} \boldsymbol{\lambda_1}) \cdot \mathrm{Cor}(\boldsymbol{\lambda_1} \xrightarrow{R^{\tilde{b}}} \boldsymbol{\lambda_2}) \\ \cdot \mathrm{Cor}(\boldsymbol{\lambda_2} \xrightarrow{R^{\tilde{c}}} \boldsymbol{\lambda_1} \oplus \boldsymbol{\lambda_2} \oplus \boldsymbol{\lambda_t}) \cdot \mathrm{Cor}(\boldsymbol{\lambda_1} \oplus \boldsymbol{\lambda_2} \oplus \boldsymbol{\lambda_t} \xrightarrow{R^{\tilde{d}}} \boldsymbol{\lambda_c}). \end{aligned} \tag{8}$$

  This matches the form under the ST setting. However, when the plaintext or ciphertext is fixed, i.e., $\mathrm{Cor}(\boldsymbol{\lambda_m} \xrightarrow{R^{\tilde{a}}} \boldsymbol{\lambda_1}) = 1$ or $\mathrm{Cor}(\boldsymbol{\lambda_1} \oplus \boldsymbol{\lambda_2} \oplus \boldsymbol{\lambda_t} \xrightarrow{R^{\tilde{d}}} \boldsymbol{\lambda_c}) = 1$, the resulting correlation $c_{\text{4-LRW1}}$ becomes larger, weakening the linear security compared to the ST case.

- For $E = E_{\text{2-LRW2}}$, the trails involve intermediate masks $\boldsymbol{\lambda_2}$, and the corresponding trail correlation is:

$$\begin{aligned} c_{\text{2-LRW2}} = \mathrm{Cor}(\boldsymbol{\lambda_m} \xrightarrow{R^{\hat{b}}} \boldsymbol{\lambda_2}) \cdot \mathrm{Cor}(\boldsymbol{\lambda_2} \xrightarrow{R^{\hat{c}}} \boldsymbol{\lambda_c}) \\ \cdot \mathrm{Cor}\left(\boldsymbol{\lambda_t} \xrightarrow{h_1, h_2, h_1 \oplus h_2} \boldsymbol{\lambda_m}, \boldsymbol{\lambda_c}, \boldsymbol{\lambda_2}\right). \end{aligned}$$

  Since $c_h \leq 1$ by definition, the ST linear security of $E_{\text{2-LRW2}}$ directly implies its security against MT linear attacks.

4.3.3 MT Algebraic Attacks

For non-zero $\boldsymbol{u}$, we focus on the algebraic degree of the ciphertext $\boldsymbol{c}$ or the plaintext $\boldsymbol{m}$ with respect to $\boldsymbol{t_u}$, i.e., $\deg(E, \boldsymbol{u})$ and $\deg(E^{-1}, \boldsymbol{u})$, respectively.

- For $E = E_{\mathsf{THF}}$, the longest algebraic degree trail during encryption follows the path $\boldsymbol{t} \to h_1 \to R^{b+c+d}$, while during decryption it follows $\boldsymbol{t} \to h_2 \to R^{-(a+b+c)}$. Therefore, the algebraic degree can be approximated as:

$$\begin{aligned} \deg(E_{\mathsf{THF}}, \boldsymbol{u}) &\approx \deg\left(R^{b+c+d} \circ h_1, \boldsymbol{u}\right), \\ \deg(E^{-1}_{\mathsf{THF}}, \boldsymbol{u}) &\approx \deg\left(R^{-(a+b+c)} \circ h_2, \boldsymbol{u}\right). \end{aligned} \tag{9}$$

  This indicates that the resistance to algebraic attacks in the MT setting also depends on the algebraic degree of the hash functions h_1 and h_2.

- For $E = E_{\text{4-LRW1}}$, we have:

$$\begin{aligned} \deg(E_{\text{4-LRW1}}, \boldsymbol{u}) &\approx \deg(R^{\tilde{b}+\tilde{c}+\tilde{d}}, \boldsymbol{u}), \\ \deg(E^{-1}_{\text{4-LRW1}}, \boldsymbol{u}) &\approx \deg(R^{-(\tilde{a}+\tilde{b}+\tilde{c})}, \boldsymbol{u}). \end{aligned} \tag{10}$$

The tweak-dependent algebraic degree in both forward and inverse directions is reduced in the MT setting. This results in weaker resistance against algebraic attacks compared to the ST scenario.

- For $E = E_{\text{2-LRW2}}$, we have the following approximations:

$$\deg(E_{\text{2-LRW2}}, \boldsymbol{u}) \approx \deg(R^{\hat{b}+\hat{c}} \circ h_1, \boldsymbol{u}),$$
$$\deg(E^{-1}_{\text{2-LRW2}}, \boldsymbol{u}) \approx \deg(R^{-(\hat{b}+\hat{c})} \circ h_2, \boldsymbol{u}).$$

These expressions indicate that the algebraic degree of the cipher under MT scenarios is not lower than that under ST scenarios. Therefore, the ST algebraic security of $E_{\text{2-LRW2}}$ inherently implies its security against algebraic attacks in the MT setting.

4.4 Comparison and Summary

We summarize the analysis results in Table 1. Rows 2–4 compare the security of the MT setting against the corresponding ST setting across three types of cryptanalytic evaluations. Row 5 presents the minimum number of rounds required to achieve the desired security level under both the ST and MT settings. Rows 6 and 7 report the encryption and decryption latencies of the secure constructions.

From the analysis, it is observed that the E_{THF} and $E_{\text{4-LRW1}}$ generally require larger values of parameters $b, c, \tilde{b}, \tilde{c}$ to achieve higher resistance to MT attacks. In contrast, for $E_{\text{2-LRW2}}$, larger values of $\hat{b}, \hat{c}$ are required to maintain security against ST attacks. As a result, we assume that the parameters $b, c, \tilde{b}, \tilde{c}, \hat{b}, \hat{c} \geq r_C$ can typically be satisfied. Based on these assumptions, we derive the encryption and decryption latencies for three ciphers, as shown in the table.

For E_{THF}, assuming that $a + b + c + d = r_S$, we begin with $a = d = 0$ and increment the values of a and d. There is likely a critical threshold where increasing a and d will still maintain resistance against both linear and algebraic attacks. However, beyond this threshold, further increases in a and d will make the construction insecure. This is because when $a = d = 0$, the construction effectively becomes equivalent to $E_{\text{2-LRW2}}$. From the table, we can observe that $E_{\text{2-LRW2}}$ already has redundancy in its resistance to MT linear and algebraic attacks. Therefore, the latency of E_{THF} has a greater potential for lower latency compared to $E_{\text{2-LRW2}}$.

The presence of hash functions in E_{THF} enhances its security relative to $E_{\text{4-LRW1}}$ under the same rounds distribution. This can be confirmed through Equations 6, 8, 9 and 10. As a result, for the same level of security, E_{THF} generally requires fewer rounds. Moreover, if the hash functions h_1 and h_2 are designed to have low-latency characteristics, especially if their latency is less than or equal to $r_C \cdot T(R)$, then E_{THF} exhibits improved latency performance. Thus, the key to constructing low-latency ciphers using the E_{THF} structure lies in selecting a low-latency 2^{-n}-AXUHF, which plays a crucial role in optimizing the overall performance of the cipher. Furthermore, we have identified a low-latency hash functions family, which will be detailed in a later section. This makes it practically feasible to construct a low-latency cipher using the THF mode.

5 Instantiation: `Blink`

In this section, we instantiate the THF mode by proposing a concrete tweakable block cipher family named `Blink`. The `Blink` family supports multiple configurations, including block sizes of 64 and 128 bits, and tweak lengths equal to one or two times the block size. For simplicity, in the following discussion we denote the block size by n and assume that the length of tweak $\boldsymbol{t}$ is fixed to τ.

Table 1: Comparison of three constructions across security and performance aspects.

Metric	E_{THF}	$E_{\text{4-LRW1}}$	$E_{\text{2-LRW2}}$
MT differential analysis	–	$\downarrow$	–
MT linear analysis	D	$\downarrow$	$\uparrow$
MT algebraic attack	D	$\downarrow$	$\uparrow$
#Rounds requirement	$a{+}b{+}c{+}d \geq r_S$ $b, c \geq r_C$	$\tilde{a}{+}\tilde{b}{+}\tilde{c}{+}\tilde{d} > r_S$ $\tilde{a}, \tilde{b}, \tilde{c}, \tilde{d} \geq r_C$	$\hat{b}{+}\hat{c} = r_S$ $\hat{b}, \hat{c} \geq r_C$
Encryption latency	$\geq r_S{\cdot}T(R){+}\Delta_a$	$> r_S \cdot T(R)$	$r_S{\cdot}T(R){+}T(h_1)$
Decryption latency	$\geq r_S{\cdot}T(R){+}\Delta_d$	$> r_S \cdot T(R)$	$r_S{\cdot}T(R){+}T(h_2)$

Legend: $\downarrow$: MT security is typically weaker than ST; $\uparrow$: stronger; –: secure in this setting; D: the security depends on the property of hash functions.
$\Delta_a = \max\{0, T(h_1) - a \cdot T(R)\}, \quad \Delta_d = \max\{0, T(h_2) - d \cdot T(R)\}$.

5.1 The Construction

`Blink`'s overall structure is illustrated in Figure 2. While based on the THF construction, `Blink` introduces several modifications compared to the instantiation E_{THF} in Section 4, to better support hardware efficiency. In particular, `Blink` adopts a reflector construction [BCG+12], which reduces the hardware footprint by exploiting structural symmetry. We next explain the notations used in Figure 2.

Each round function consists of five operations: an S-box layer (`S`), a MixColumn layer (`M`), round key addition (`AK`), round constant addition (`AC`), and a shuffle layer (`P`). The round function is:

$$\mathtt{R} = \mathtt{P} \circ \mathtt{AC} \circ \mathtt{AK} \circ \mathtt{M} \circ \mathtt{S}.$$

We denote $\mathtt{MK}_{\boldsymbol{k}}(\boldsymbol{x}) = \mathtt{M}(\boldsymbol{x}) \oplus \boldsymbol{k}$, and $\widehat{\boldsymbol{z}} = \mathtt{M}(\boldsymbol{z})$. Inverses are marked with overlines, e.g., $\overline{\mathtt{P}}$. Since `S` and `M` are involutive, the inverse of round function is thus:

$$\overline{\mathtt{R}} = \mathtt{S} \circ \mathtt{MK}_{\widehat{\boldsymbol{rk} \oplus \boldsymbol{rc}}} \circ \overline{\mathtt{P}}.$$

This shows that the structure depicted in Figure 2 supports a reflection property.

`Blink` corresponds to the four permutations in THF as:

- π_1: $\mathtt{M} \circ \mathtt{S} \circ \mathtt{R}^a(\bullet \oplus \boldsymbol{w}_1)$,
- π_2: $\mathtt{M} \circ \mathtt{S} \circ \mathtt{R}^b \circ \mathtt{P}$,
- π_3: $\overline{\mathtt{P}} \circ \overline{\mathtt{R}}^b \circ \mathtt{S}$,
- π_4: $\overline{\mathtt{R}}^a \circ \mathtt{S} \circ \mathtt{M}(\bullet) \oplus \boldsymbol{w}_2$.

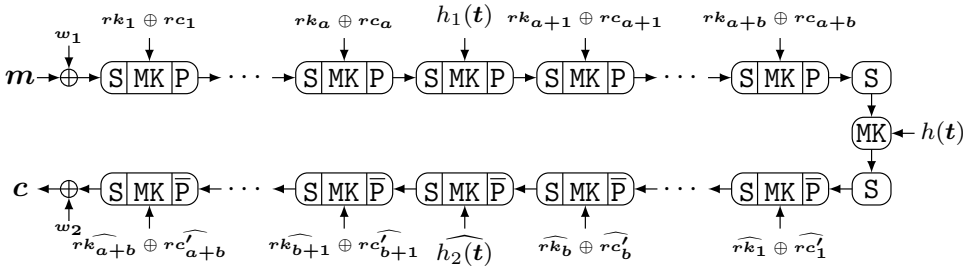

Figure 2: The overview of `Blink`

5.2 The Round Function

The internal state of `Blink` is organized as a two-dimensional array, consisting of 4 rows and $n/16$ columns, where each cell represents a nibble (4 bits). The state $\boldsymbol{s_{n/4-1}} \| \cdots \| \boldsymbol{s_1} \| \boldsymbol{s_0}$ can be visualized as follows:

$$\begin{bmatrix} \boldsymbol{s_0} & \boldsymbol{s_1} & \cdots & \boldsymbol{s_{n/16-1}} \\ \boldsymbol{s_{n/16}} & \boldsymbol{s_{n/16+1}} & \cdots & \boldsymbol{s_{n/8-1}} \\ \boldsymbol{s_{n/8}} & \boldsymbol{s_{n/8+1}} & \cdots & \boldsymbol{s_{3n/16-1}} \\ \boldsymbol{s_{3n/16}} & \boldsymbol{s_{3n/16+1}} & \cdots & \boldsymbol{s_{n/4-1}} \end{bmatrix},$$

Each operation in round function `R` updates internal states as follows.

- `S`: The 4-bit S-box S is applied to each cell in parallel. Its specification in hexadecimal is shown in the following table. Additionally, the S-box is an involution, meaning that its inverse is equal to itself.

x	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
$S(x)$	1	0	9	3	8	5	e	7	4	2	c	b	a	f	6	d

- `M`: A diffusion matrix M is multiplied to each column. Namely,

$$\left[\boldsymbol{s_j}, \boldsymbol{s_{j+n/16}}, \boldsymbol{s_{j+n/8}}, \boldsymbol{s_{j+3n/16}}\right]^T \leftarrow M \left[\boldsymbol{s_j}, \boldsymbol{s_{j+n/16}}, \boldsymbol{s_{j+n/8}}, \boldsymbol{s_{j+3n/16}}\right]^T,$$

  where $0 \leq j < n/16$, and M is the involutory matrix in `Midori`:

$$M = \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}.$$

- `AK`: The round key is XORed to the state.

- `AC`: The round constant is XORed to the state. The specific values for the round constants $\boldsymbol{rc_i}$ and $\boldsymbol{rc'_i}$ are provided in Appendix D.

- `P`: Each cell in the state is shuffled according to the permutation P. Namely,

$$\left[\boldsymbol{s_0}, \boldsymbol{s_1}, \cdots, \boldsymbol{s_{n/4-1}}\right] \leftarrow \left[\boldsymbol{s_{P[0]}}, \boldsymbol{s_{P[1]}}, \cdots, \boldsymbol{s_{P[n/4-1]}}\right].$$

  Specifically, for 64-bit block

$$P = [0, 5, 11, 10, 1, 6, 4, 13, 2, 12, 9, 15, 3, 7, 14, 8],$$

  and for 128-bit block,

$$\begin{aligned} P = [&5, 12, 4, 1, 17, 9, 10, 16, 28, 14, 21, 22, 11, 27, 8, 13, \\ &2, 25, 18, 3, 30, 6, 19, 20, 0, 23, 24, 31, 7, 15, 29, 26]. \end{aligned}$$

5.3 The Hash Function

We define the map

$$h_T : \{0,1\}^\tau \to \{0,1\}^n, \qquad h_T(\mathbf{t}) = T \cdot \boldsymbol{t},$$

where $T \in \{0,1\}^{n\times\tau}$ and $\boldsymbol{t} = (t_0, t_1, \ldots, t_{\tau-1})^\top \in \{0,1\}^\tau$. Here t_0 denotes the least significant bit of $\boldsymbol{t}$. The hash family based on Toeplitz matrices is then defined as:

$$\mathcal{H} = \{h_T \mid T \text{ is an keyed } n \times \tau \text{ Toeplitz matrix}\}.$$

A Toeplitz matrix is a matrix in which each element on the same diagonal is identical, meaning it is uniquely determined by the values of $n + \tau - 1$ diagonals. Specifically, the Toeplitz matrix T with $\boldsymbol{k} = (k_0, k_1, \ldots, k_{n+\tau-2})$ is defined as

$$T = \begin{bmatrix} k_{n-1} & k_n & \cdots & k_{n+\tau-2} \\ k_{n-2} & k_{n-1} & \cdots & k_{n+\tau-3} \\ \vdots & \vdots & \ddots & \vdots \\ k_0 & k_1 & \cdots & k_{\tau-1} \end{bmatrix}. \tag{11}$$

In the `Blink`, h_1 and h_2 are randomly selected from the hash family $\mathcal{H}$, and they are independently determined by the $(n + \tau - 1)$-bit keys $\boldsymbol{k_1}$ and $\boldsymbol{k_2}$, respectively. The function h is defined as the sum of h_1 and h_2, i.e.,

$$h(\boldsymbol{t}) = h_1(\boldsymbol{t}) \oplus h_2(\boldsymbol{t}).$$

5.4 The Key Schedule

The master key $\boldsymbol{k}$ is the concatenation of whitening keys $\boldsymbol{w_1}$, $\boldsymbol{w_2}$, and round keys $\boldsymbol{rk_i}$:

$$\boldsymbol{k} = \boldsymbol{rk_{a+b}} \| \cdots \| \boldsymbol{rk_1} \| \boldsymbol{w_2} \| \boldsymbol{w_1},$$

with a total length of $(a + b + 2)n$ bits.

Let $\boldsymbol{k'}$ be a rearrangement of $\boldsymbol{k}$, where each bit $k'_i = k_{11\cdot i \bmod (a+b+2)n}$. The key $\boldsymbol{k_2}\|\boldsymbol{k_1}$, used for generating the hash functions h_1 and h_2, is derived from the least significant $2n + 2\tau - 2$ bits of $\boldsymbol{k'}$.

5.5 Security Claims

According to different security requirements, the number of rounds for each version is summarized in Table 2. Similarly to `QARMAv2` [ABD+23], we restrict the data volume per master key to at most 2^{56} blocks for the 64-bit variant and 2^{80} blocks for the 128-bit variant. This setting is consistent with existing recommendations, such as those from the NIST call for lightweight cryptographic algorithms. We also believe that these limits are ample for real-world applications.

In addition, we require that the tweak value $\boldsymbol{t}$ is non-zero. When $\boldsymbol{t} = \boldsymbol{0}$, the derived values $h_1(\boldsymbol{t})$, $h_2(\boldsymbol{t})$, and $h(\boldsymbol{t})$ all degenerate to $\boldsymbol{0}$, which will significantly reduce the security margin [CGZ25]. This constraint is trivial to satisfy in practice: one may reserve a single bit (e.g., the MSB) to 1 and combine the remaining bits with a nonce or physical address. When a counter is used as the tweak, counting simply starts from 1 instead of 0.

Remark. We do not claim the security of `Blink` against related-key attacks.

6 Design Rationale for `Blink`

This section outlines the design rationale behind the `Blink` family. Its primary goal is to instantiate the THF construction to demonstrate its low-latency potential, with a particular focus on memory encryption scenarios.

Table 2: Security claims and parameter choices.

Version	n	τ	a	b	Key Size	Time	Data
`Blink-64a`	64	64	2	3	448	2^{128}	2^{56}
`Blink-64b`	64	128	2	3	448	2^{128}	2^{56}
`Blink-128a`	128	128	3	3	1024	2^{128}	2^{80}
`Blink-128b`	128	256	3	3	1024	2^{128}	2^{80}
`Blink-128A`	128	128	3	5	1280	2^{256}	2^{80}
`Blink-128B`	128	256	3	5	1280	2^{256}	2^{80}

6.1 Design of the Hash Function

As described in Section 4.4, the key to constructing a low-latency tweakable block cipher via the THF mode is the identification of an efficient AXUHF. Universal hash functions have a wide range of applications, including message authentication codes [WC81, BHK+99, CV03], authenticated encryption schemes [MV04], and tweakable block ciphers [LST12b, JLM+19a, LL18]. Most constructions rely on basic mathematical operations, such as modular multiplication, polynomial multiplication, matrix multiplication, or finite field multiplication [CW79, MNT90, Sti92, Kra94, BHK+99, AL11, Nan14, GFAD23]. Recently, AES-based hash functions have been proposed that leverage hardware acceleration to improve efficiency [BBL+24].

We observe that matrix-based universal hash functions exhibit superior latency characteristics. In particular, the family

$$\mathcal{H}_3 = \{h_A \mid A \text{ is a keyed } n \times \tau \text{ matrix}\}$$

can be implemented with latency of only $\lceil \log_2(\tau) \rceil \cdot T(\texttt{XOR/NXOR}) + T(\texttt{NAND})$, yielding a circuit depth (as defined in Definition 1) of $2\lceil \log_2(\tau) \rceil + 1$. This construction achieves 2^{-n}-almost XOR universality, with key size $n \cdot \tau$ bits.

Based on this, [MNT90] showed that constraining A to be a Toeplitz matrix preserves the 2^{-n}-AXU property while reducing the key size to $n+\tau-1$ bits. Another construction, proposed by [Kra94], uses an n-bit linear feedback shift register (LFSR) to hash messages, but this limits the key size to n and requires the initialization to be the coefficients of an irreducible polynomial of degree n. Given that a 256-bit tweak is sufficient for most applications, we adopt the Toeplitz matrix-based construction due to its efficient key size and relaxed structural constraints, which simplify key generation.

Recall that the hash family based on Toeplitz matrices is:

$$\mathcal{H} = \{h_T \mid T \text{ is a keyed } n \times \tau \text{ Toeplitz matrix}\},$$

with the structure of the matrix given in Equation 11.

The circuit depth of any $h_T \in \mathcal{H}$ is equal to that of $h_A \in \mathcal{H}_3$, namely,

$$\text{depth}(h_T) = 2\lceil \log_2(\tau) \rceil + 1.$$

For typical values of τ, such as 64, 128, and 256, the corresponding depths are 13, 15, and 17, respectively. This low latency makes the Toeplitz-based hash function particularly suitable for embedding within the THF mode to construct low-latency tweakable block ciphers.

In addition to its 2^{-n}-AXU property, we examine the correlation c_h (as defined in Equation 7) and the algebraic degree to evaluate the resistance offered by h_T for E_{THF} against MT linear and algebraic attacks.

For variable tweaks and $\boldsymbol{\lambda_t} \neq \mathbf{0}$, we must have $\boldsymbol{\lambda_1} \oplus \boldsymbol{\lambda_2} \neq \mathbf{0}$ or $\boldsymbol{\lambda_3} \oplus \boldsymbol{\lambda_2} \neq \mathbf{0}$. The linear approximation holds with probability 1 with probability 2^{-n}, and 1/2 otherwise. Consequently, the expected correlation satisfies:

$$c_h = 2\left(2^{-n} + \frac{1-2^{-n}}{2}\right) - 1 = 2^{-n},$$

offering exponential resistance to MT linear attacks.

Furthermore, since h_T is linear in the tweak input $\boldsymbol{t}$, its algebraic degree in the variable $\boldsymbol{t_u}$ is 1, indicating negligible contribution to algebraic attacks. Thus, h_T is well suited for extending low-latency block ciphers (where algebraic attacks are not a primary threat) into tweakable ones with minimal latency overhead.

6.2 Design of Round Function

`Blink` is designed for memory encryption, where the read/write unit corresponds to a cache line, typically 512 or 1024 bits. Unlike block ciphers used in modes like CTR, where only encryption is invoked, tweakable block ciphers are typically required to support both encryption and decryption. Therefore, to align with typical cache line boundaries, the block size in `Blink` is restricted to commonly used values as 64 or 128 bits.

Among existing low-latency ciphers with these block sizes, `Midori` [BBI+15] demonstrates strong resistance against differential and linear attacks. Its round function has been adopted in `Mantis` [BJK+16], and its MixColumn layer reused in the permutation `Orthros` [BIL+21]. Building on this, `Blink` designs its round function based on the structure of `Midori`.

As `Blink` adopts a reflection structure, minimizing the latency of the round function `R` and its inverse $\overline{\mathtt{R}}$ is essential for overall performance. Each component of the round function is thus optimized for both forward and inverse computations, ensuring low latency in both directions.

S-boxes. Since `Blink` uses 64- or 128-bit blocks, we consider 4- or 8-bit S-boxes accordingly. For 8-bit low-latency S-boxes proposed in the literature [Ras22], the best cryptographic properties achieved are linearity 128 and uniformity 64. However, these properties do not demonstrate an improvement over 4-bit S-boxes in terms of cryptographic strength. By opting for 4-bit S-boxes, the number of active S-boxes can propagate over a larger space, offering stronger resistance to differential and linear attacks. This advantage has also been highlighted in the design of `QARMAv2` [ABD+23].

Therefore, we adopt a 4-bit S-box for both the 64-bit and 128-bit variants of `Blink`, guided by the following security criteria: (a) bijectivity; (b) differential uniformity of 4; (c) linearity of 8; (d) full diffusion, i.e., each input bit influences all output bits.

To minimize latency, we target an S-box and its inverse with a depth of at most 3.5. Since involutory S-boxes are their own inverses, their forward and inverse depths are identical. Thus, we focus on identifying involutory S-boxes that satisfy the above criteria, simplifying the search process and ensuring low-latency implementation in both encryption and decryption.

In the study by Rasoolzadeh [Ras22], three *extended bit permutation equivalence* classes (as defined in Appendix A) of 4-bit S-boxes were identified that meet the following criteria: involutory property, differential uniformity of 4, linearity of 8, and a depth of 3.5[2]. Among these, only the first equivalence class guarantees full diffusion. Notably, each output bit of the representative S-box in this class achieves the maximum algebraic degree of 3 (see Appendix A), effectively compensating for the low algebraic degree of the Toeplitz matrix-based hash function. Therefore, an involutory S-box from this class is selected for `Blink`.

[2]The depth corresponds to a latency complexity of 3, as defined in [Ras22].

Its implementation is shown below, where $\overline{x_i}$ denotes $\texttt{NOT}(x_i)$.

$$y_0 = \big((\overline{x_2}\ \texttt{NAND}\ \overline{x_1})\ \texttt{NOR}\ \overline{x_0}\big)\ \texttt{NOR}\ \big((x_3\ \texttt{NOR}\ x_2)\ \texttt{NOR}\ x_0\big)$$
$$y_1 = \big((\overline{x_2}\ \texttt{NAND}\ \overline{x_0})\ \texttt{NAND}\ (\overline{x_3}\ \texttt{NAND}\ \overline{x_1})\big)\ \texttt{NOR}\ \big((x_3\ \texttt{NAND}\ x_0)\ \texttt{NOR}\ (x_2\ \texttt{NAND}\ x_1)\big)$$
$$y_2 = \big((\overline{x_1}\ \texttt{NAND}\ \overline{x_0})\ \texttt{NAND}\ x_2\big)\ \texttt{NAND}\ \big((x_2\ \texttt{NOR}\ x_0)\ \texttt{NAND}\ x_3\big)$$
$$y_3 = \big((\overline{x_0}\ \texttt{NAND}\ x_2)\ \texttt{NOR}\ (x_3\ \texttt{NAND}\ x_1)\big)\ \texttt{NOR}\ \big((\overline{x_3}\ \texttt{NAND}\ x_0)\ \texttt{NAND}\ (\overline{x_2}\ \texttt{NAND}\ \overline{x_1})\big).$$

Linear Layer. We adopt 4×4 almost MDS (maximum distance separable) binary matrix M used in Midori [BBI+15], which offers significantly lower latency compared to MDS matrices. Additionally, M is an involutory matrix, ensuring identical latency for both forward and inverse computations.

The cell permutation for shuffle layer is primarily designed to follow the wide-trail strategy [DR02], which requires mapping cells from the same column to distinct columns, as stated in Condition 1. Consequently, after the permutation, each column must consist of cells originating from different input columns.

Condition 1. *For each column* $[\boldsymbol{s_i}, \boldsymbol{s_{i+4}}, \boldsymbol{s_{i+8}}, \boldsymbol{s_{i+12}}]$ *in the* $4 \times 4 \times \frac{n}{64}$ *array, after applying the shuffle layer, these cells will be mapped to 4 cells in different columns, where* $i \in \big\{j + 16l \mid j \in \{0, \ldots, 3\}, l \in \{0, \ldots, \frac{n}{64} - 1\}\big\}$.

For the 64-bit block size, the design of the shuffle layer in conjunction with the diffusion matrix M was thoroughly analyzed by Alfarano *et al.* [ABI+18]. Based on the classification in Appendix C.1 of [ABI+18], we select a permutation from the first class that satisfies Condition 1 and ensures full diffusion within three rounds in both forward and backward directions. Furthermore, this permutation achieves the optimal lower bound on the number of active S-boxes over the first eight rounds.

For the 128-bit block size, we introduce an additional requirement, Condition 2, to further improve diffusion. Together with Condition 1, it ensures that each output cell is influenced by exactly 23 input cells after three rounds, in both forward and backward directions. As shown in Appendix B, increasing the number of contributing input columns per output cell enhances overall diffusion—motivating the introduction of Condition 2. Notably, among all permutations satisfying Condition 1, the maximum attainable minimum number of influencing input cells per output cell is 23.

Condition 2. *For the case* $n = 128$*, after applying the permutation three times, each output cell should be influenced by input cells from at least 7 distinct columns, and each input cell should contribute to output cells in at least 7 distinct columns.*

Given the vast search space, randomly identifying permutations satisfying both conditions is infeasible. To overcome this, we formulated the conditions as MILP constraints and used the Gurobi solver to systematically explore a set of feasible candidates. For each candidate, we computed the lower bounds on the number of active S-boxes under both ST differential and linear attacks, and selected the permutation with the best bounds.

Round Constants. Considering that `Blink-64` shares a similar round function with `Midori-64`, it is worth noting that `Midori-64` are vulnerable to invariant subspace attacks [BCLR17, GJN+16]. One of the main reasons for this vulnerability is that the round constants in `Midori-64` are set to 0 or 1 at the cell level. Combined with the specific properties of its S-box and diffusion matrix, this design enables the existence of invariant subspaces.

To address this issue in the design of `Blink`, the round constants $\boldsymbol{rc_i}$ and $\boldsymbol{rc'_i}$ in the 128-bit block version are derived from the fractional part of $\pi = 3.14159\ldots$, following the approach used in `PRINCE` [BCG+12]. For the 64-bit block version, each round constant is defined as the least significant 64 bits of the corresponding constant used in the 128-bit version. The detailed values of these constants are provided in Appendix D.

6.3 Choice of the Number of Rounds

The latency of Toeplitz hash function family is approximately $2\lceil \log_2(\tau) \rceil + 1$. For specific values of τ, such as 64, 128, and 256, the corresponding depths are 13, 15, and 17, respectively. Considering the delay overhead introduced by high fan-out, and given that the depth of a single round function is 7.5, the overall latency of the hash function is roughly equivalent to that of three to four rounds.

As discussed in Section 4, increasing the parameter a improves parallelism and helps hide the latency of the hash function. However, a larger value of a may also increase the total number of rounds required to achieve a sufficient security level. Therefore, we selected parameters a and b by jointly considering both parallelizability and the results of our security analysis, aiming to achieve near-optimal latency.

Notably, due to the strong resistance of `Blink` against ST algebraic attacks, fewer rounds are sufficient to meet the ST security requirement. As a result, we selected a such that the latency of computing $h(t)$ is roughly hidden by the parallel execution of the encryption rounds, resulting in nearly the same total number of rounds required for both ST and MT security.

6.4 Design of Key Schedule

Unlike most symmetric-key encryption algorithms, the master key in `Blink` exceeds the targeted security level, with a length of up to 1280 bits. A longer key eliminates the need for a complex key schedule design and ensures independence between the subkeys $\boldsymbol{k_1}$ and $\boldsymbol{k_2}$ used in hash generation. Moreover, a long key provides enhanced security compared to shorter ones by increasing the complexity of key-recovery attacks.

The primary application scenario of `Blink` is memory encryption for PC-class systems or flagship mobile devices. In such environments, the key generation and storage mechanisms follow an approach similar to the Intel SGX scheme [Gue16]. Specifically, the master key is generated and managed by a trusted firmware module during the system boot process.

Large master keys can be efficiently produced using hardware random number generators (RNGs), eliminating the need for a dedicated key schedule circuit. This approach conserves chip area and reduces the energy consumption associated with frequent key schedule operations. Furthermore, the master key has a long usage period, limited only by the data bounds (2^{56} for Blink-64 and 2^{80} for Blink-128), making the latency of invoking hardware RNGs negligible. After generation, the trusted firmware module programs the master key into hardware-protected MEE registers. The register space required for a master key of up to 1280 bits remains modest relative to the silicon budgets of mainstream CPUs.

In contrast to `Midori-64`, `Blink` introduces correlations only between every two round keys at non-symmetric positions. Combined with the use of random round constants, this design mitigates the risk of invariant subspace attacks, ensuring that the vulnerabilities present in `Midori-64` do not compromise `Blink`.

Furthermore, the key arrangement at non-symmetric positions is deliberately designed to avoid the α-reflection property ($F_{\boldsymbol{k}} = F^{-1}_{\boldsymbol{k} \oplus \boldsymbol{\alpha}}$) in order to prevent potential attacks that exploit this structural symmetry. Both encryption and decryption require the use of $\boldsymbol{rk_i}$ and $\hat{\boldsymbol{rk_i}}$ in a specific order. During encryption, the round keys are used in the following sequence:

$$\boldsymbol{rk_1}, \ldots, \boldsymbol{rk_{a+b}}, \hat{\boldsymbol{rk_1}}, \ldots, \hat{\boldsymbol{rk_{a+b}}}.$$

For decryption, the order is reversed:

$$\boldsymbol{rk_{a+b}}, \ldots, \boldsymbol{rk_1}, \hat{\boldsymbol{rk_{a+b}}}, \ldots, \hat{\boldsymbol{rk_1}}.$$

The randomness in the choice of the round constants ensures that $\forall i$,

$$\boldsymbol{rk_i} \oplus \boldsymbol{rc_i} \oplus \boldsymbol{rk_{a+b-i}} \oplus \boldsymbol{rc'_{a+b-i}} \neq \boldsymbol{rk_{a+b-i}} \oplus \boldsymbol{rc_{a+b-i}} \oplus \boldsymbol{rk_i} \oplus \boldsymbol{rc'_i},$$

preventing the existence of special weak keys that could cause `Blink` to satisfy the α-reflection property.

7 Security Analysis

7.1 Differential Analysis

We employed the MILP approach for differential analysis [BS91]. The obtained results, including bounds on the active S-boxes numbers, are summarized in Table 3, where "full cipher" refers to the full reflective construction. Each round containing an S-box layer is counted as a single round in the overall round count.

For `Blink-64`, since the maximum differential probability of the S-box in `Blink` is 2^{-2}, the maximum differential probability of 10-round "full cipher" differential characteristic is at most 2^{-100}. Hence, we conclude that when the data does not exceed 2^{56}, it is highly unlikely to distinguish 10 rounds of `Blink-64` without any key constraints.

Furthermore, due to the fact that after applying `R` or its inverses three times, each output cell is influenced by all input cells and any three consecutive rounds keys are independent, the output of the third encryption or decryption round is affected by $(16 + 9 + 3) \times 4 = 112$ bits of the key, while the second encryption or decryption round is affected by $(9 + 3) \times 4 = 48$ bits of the key. Consequently, for the 14-round `Blink-64`, only a 10-round distinguisher could potentially be useful to recover the full key. Thus, we think `Blink-64` is secure against differential attacks.

Similarly, for `Blink-128`, when the data does not exceed 2^{80}, it is highly unlikely to distinguish 10 rounds of `Blink-128` without any key constraints. The output of the third / fourth encryption or decryption round is affected by $(23 + 9 + 3) \times 4 = 140$ / $(31 + 23 + 9 + 3) \times 4 = 264$ bits of the key, respectively. Thus, we think `Blink-128` is also secure against differential attacks.

Weak-key Differential Attacks. Recent work [CGZ25] analyzes the special case where the tweak is fixed to zero and constructs superboxes exploiting this degenerate condition. Under this assumption, for `Blink-64a/b` with a 448-bit key, they obtain a 10-round differential distinguisher for up to 2^{442} weak keys with probability $2^{-50.42}$. For `Blink-128a/b` with a 1024-bit key, they derive a 10-round distinguisher for up to 2^{1010} weak keys with probability $2^{-68.83}$, which can be extended to a 12-round multiple-differential distinguisher with probability $2^{-96.83}$. Based on the weak tweak-key distinguisher, they further mount a 13-round key-recovery attack on `Blink-64`, recovering the full 448-bit master key with time complexity $2^{112.15}$ and requiring 2^{56} chosen plaintexts.

These results crucially rely on the degenerate tweak value $\boldsymbol{t} = \mathbf{0}$, for which the hash functions h_1, h_2, h collapse to zero and lose their intended key/tweak mixing properties. The `Blink` specification excludes this corner case by design: as described in Section 5.5, the tweak space is restricted to non-zero values, and standard padding or counter-offset conventions make this requirement trivial to satisfy in practice.

When $\boldsymbol{t} \neq \mathbf{0}$, the values of the hash functions are key dependent and cannot be predetermined, preventing the structural collapses exploited in [CGZ25]. Consequently, extending their weak-tweak analysis to the full tweak domain would require substantially stronger assumptions and distinguishers far beyond the allowable data limits.

We therefore conclude that, under the specified tweak space $\mathbb{F}_2^\tau \setminus \{\mathbf{0}\}$, `Blink` remains secure against differential attacks, including weak-key scenarios.

Table 3: The lower bounds on the number of active S-boxes for `Blink`

Block Size	Half Cipher								Full Cipher					
	1	2	3	4	5	6	7	8	2	4	6	8	10	12
64	1	4	7	16	23	30	35	38	4	16	24	32	50	64
128	1	4	7	16	24	36	48	58	4	16	24	40	64	-

7.1.1 MT Differential Attacks

For `Blink-64` / `Blink-128`, h_1 and h_2 both belong to 2^{-64}-AXUHF / 2^{-128}-AXUHF, respectively. Therefore, under the data limitations of 2^{56} / 2^{80}, `Blink-64` and `Blink-128` are sufficiently resistant to MT differential attacks.

7.2 Impossible Differential Attacks

We explored the impossible differential trails for the center structure, considering r_1-rounds of the upper half and r_2-rounds of the lower half. Inspired by [ST17], we constrained the differential model such that the input and output have only one active cell. By determining the feasibility of the model, we verify whether the input-output differential pair is valid. The final results are summarized in Table 4, where '✓' indicates the existence of an $r_1 + r_2$ impossible differential trail, and 'blank' denotes that for any difference pair with a single active cell in the input and output, a valid trail exists.

For `Blink-64`, the longest impossible differential trail spans 7 rounds, while it spans 9 rounds for `Blink-128`. Since we have employed multiple different round keys, similar to differential analysis, these distinguishers are insufficient to pose a threat to the security of `Blink`.

Table 4: Existence of $r_1 + r_2$-round impossible difference

	`Blink-64`							`Blink-128`									
r_2 / r_1	0	1	2	3	4	5	6	0	1	2	3	4	5	6	7	8	9
0	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
1	✓	✓	✓	✓	✓	✓		✓	✓	✓	✓	✓	✓	✓	✓	✓	
2	✓	✓	✓	✓	✓	✓		✓	✓	✓	✓	✓	✓	✓	✓		
3	✓	✓	✓	✓				✓	✓	✓	✓	✓	✓	✓			
4	✓	✓	✓					✓	✓	✓	✓	✓	✓				
5	✓	✓	✓					✓	✓	✓	✓	✓					
6	✓							✓	✓	✓	✓						
7								✓	✓	✓							
8								✓	✓								
9								✓									

7.2.1 MT Impossible Differential Attacks

Since h_1 and h_2 are 2^{-n}-AXUHF, when the difference of tweak is non-zero, the value of output difference is difficult to be determined. Therefore, we believe that Blink is secure against MT impossible differential attacks.

7.3 Linear Analysis

Since the diffusion matrix M is symmetric and involutory, and the shuffle layer is cell-based, Table 3 also presents the lower bounds on the number of active S-boxes for linear characteristics. Given that the linearity of the S-box is 8, its maximum squared correlation is 2^{-2}. Similar to differential analysis, we conclude that `Blink-64` and `Blink-128` are secure against linear attacks.

7.3.1 MT Linear Attacks

Given that for variable tweaks and non-zero $\boldsymbol{\lambda_t}$, we have $c_h = 2^{-n}$, as defined in Equation 7, and by the linear analysis of E_{THF}, we conclude that `Blink` achieves security against MT linear attacks.

7.4 Integral Cryptanalysis

In [Tod15], Todo introduced the division property as a generalization of the integral property, providing a powerful tool for cryptanalysis. To evaluate the division property of `Blink`, we formulated its propagation into SAT models [SWW17] and utilized the open-source solver CaDiCaL [Bie19] to solve the models. For `Blink-64`, our analysis identified that there exists at least one division trail over 7 rounds, propagating each input with 63 active bits to each output with 1 active bit. This indicates that the algebraic degree of each output bit after 7 rounds nearly reaches the theoretical maximum value of 63, as it is challenging to eliminate all terms of degree 63 through XOR operations. Similarly, for `Blink-128`, division trails were discovered over 9 rounds for each inputs consisting of 127 active bits.

These findings highlight that both `Blink-64` and `Blink-128` exhibit strong resistance against ST / MT integral attacks.

7.5 Meet-in-the-Middle Attacks

Thanks to the use of a long master key, any four consecutive round keys in `Blink` are independent. For `Blink-64`, the output of the third and fourth encryption or decryption rounds is influenced by 112 and 176 bits of the key, respectively. For `Blink-128`, the output of the third and fourth encryption or decryption rounds is influenced by 140 and 264 bits of the key, respectively. Therefore, we conclude that long-round meet-in-the-middle attacks are unlikely to be effective.

7.6 Invariant Attacks

In [BCLR17], Beierle *et al.* proposed a method for proving resistance against invariant attacks, including the invariant subspace attack [LAAZ11, LMR15, GJN+16] and the nonlinear invariant attack [TLS16]. They found that the attack requires the existence of an invariant for the substitution layer whose linear space is invariant under the linear layer and contains all differences between the round keys.

In many ciphers, the round keys are derived in the form $\boldsymbol{rk}_i = \boldsymbol{k} \oplus \boldsymbol{rc}_i$. Since the differences of the round constants are known, the analysis can be reduced to studying the set of differences between the round constants, denoted as D. The smallest linear subspace invariant under the linear layer `L` is defined as

$$W_{\mathtt{L}}(D) := \sum_{\boldsymbol{c} \in D} \langle \mathtt{L}^i(\boldsymbol{c}), i \geq 0 \rangle = \sum_{\boldsymbol{c} \in D} W_{\mathtt{L}}(\boldsymbol{c}),$$

where $\mathtt{L} = \mathtt{P} \circ \mathtt{M}$ in `Blink`, and $W_{\mathtt{L}}(D)$ can be computed by Algorithm 1.

Algorithm 1 Computing $W_{\mathtt{L}}(D)$

Require: list of differences D, linear layer `L` as a matrix
Ensure: the subspace $W_{\mathtt{L}}(D)$

$R \leftarrow$ an empty list
$k \leftarrow$ the multiplicative order of `L`
for all $\boldsymbol{c}$ in D **do**
  for all j from 0 to $k-1$ **do**
    Add $\mathtt{L}^j(\boldsymbol{c})$ into R
  end for
end for

return the span of R

However, `Blink` employs many distinct round keys, and the differences of many round keys are not fully known, resulting in a smaller set D. For `Blink`, D is defined as

$$D := \{\boldsymbol{rc}_i \oplus \boldsymbol{rc}'_i \mid i \in \{1, \ldots, a+b\}\}.$$

In the case of `Blink-64`, the dimension of $W_{\mathtt{L}}(D)$ is measured to be 30, which by itself may appear insufficient to conclusively demonstrate resistance against invariant attacks. However, considering the introduction of a 448-bit master key, for any weak-key subspace of size 2^{-128}, 320 bits of the key material remain unknown. This implies that the differences between round keys still contain at least 256 bits of uncertainty. When these unknown differences are incorporated into D, the resulting $W_{\mathtt{L}}(D)$ becomes unpredictable. Furthermore, since the theoretical maximum dimension of $W_{\mathtt{L}}(D)$ is 64, no nontrivial subspace can fully capture all of these unknown differences. Therefore, it is reasonable to conclude that the existence of a weak-key subspace larger than 2^{-128} that could facilitate an invariant attack is highly unlikely.

In contrast, for `Blink-128a` and `Blink-128b`, the dimension of $W_{\mathtt{L}}(D)$ is 124. Using the same approach from [BCLR17], we verified that no non-trivial invariants are present in these instances either.

8 Hardware Evaluation

This section evaluates the minimal latency of a fully unrolled `Blink` circuit in hardware and compares it against other low-latency ciphers, including block ciphers `PRINCE` [BCG+12], `Midori` [BBI+15] and `SPEEDY` [LMMR21], and tweakable block ciphers `QARMA` [Ava17], `QARMAv2` [ABD+23], and `Mantis` [BJK+16]. Results, summarized in Table 5, were obtained by synthesizing fully unrolled circuits using the NanGate 45 nm, NanGate 15 nm, and ASAP 7 nm standard cell libraries. For all tests, synthesis with the NanGate 45 nm library was performed using the smallest achievable clock period to assess peak performance. In contrast, the NanGate 15 nm and ASAP 7 nm libraries employed slightly relaxed clock periods to reduce synthesis and evaluation time and to obtain results that more closely reflect realistic operating conditions.

When implemented in hardware, `Blink`'s reflection structure allows for identical encryption and decryption circuits. Multiplexers select the appropriate key based on the encryption/decryption control signal. To minimize latency, we directly implemented the composite operation `MK` as shown in Figure 2. For the three round keys derived from the hash functions, we first computed $h_1(\boldsymbol{t})$ and $h_2(\boldsymbol{t})$, then applied the `M` operation to obtain $\widehat{h_1(\boldsymbol{t})}$ and $\widehat{h_2(\boldsymbol{t})}$. The round keys $h(\boldsymbol{t})$ and $\widehat{h(\boldsymbol{t})}$ were computed as $h_1(\boldsymbol{t}) \oplus h_2(\boldsymbol{t})$ and

Table 5: Latency evaluations of various block ciphers

Cipher	Block Size	Tweak Length	Key Length	Security Level (D, T)	Minimal Latency (ns) Nangate 45 nm	Nangate 15 nm	ASAP 7 nm
PRINCE	64	-	128	$2^n, 2^{127-n}$	2.63	0.832	1.164
Midori64	64	-	128	$2^{128}, 2^{128}$	4.69	0.972	1.900
Midori128	128	-	128	$2^{128}, 2^{128}$	6.06	1.015	2.421
SPEEDY-6-192	192	-	192	$2^{128}, 2^{128}$	2.54	0.585	0.965
MANTIS	64	64	128	$2^n, 2^{126-n}$	3.28	0.964	1.450
QARMA-64-σ_0	64	64	128	$2^n, 2^{128-n}$	3.44	0.926	1.521
QARMAv2$_7$-64	64	64	128	$2^{56}, 2^{128}$	3.53	0.847	1.576
QARMAv2$_9$-64	64	128	128	$2^{56}, 2^{128}$	4.40	0.971	1.972
QARMAv2$_9$-128	128	128	128	$2^{80}, 2^{128}$	4.51	0.972	2.064
QARMAv2$_{11}$-128	128	256	128	$2^{80}, 2^{128}$	5.30	1.058	2.465
QARMAv2$_{13}$-128	128	128	256	$2^{80}, 2^{256}$	6.10	1.236	2.855
QARMAv2$_{15}$-128	128	256	256	$2^{80}, 2^{256}$	6.99	1.357	3.249
Blink-64a	64	64	448	$2^{56}, 2^{128}$	3.11	0.673	1.145
Blink-64b	64	128	448	$2^{56}, 2^{128}$	3.12	0.681	1.145
Blink-128a	128	128	1024	$2^{80}, 2^{128}$	3.55	0.814	1.308
Blink-128b	128	256	1024	$2^{80}, 2^{128}$	3.55	0.811	1.309
Blink-128A	128	128	1280	$2^{80}, 2^{256}$	4.43	0.951	1.628
Blink-128B	128	256	1280	$2^{80}, 2^{256}$	4.39	0.949	1.627

$\widehat{h_1(\boldsymbol{t})} \oplus \widehat{h_2(\boldsymbol{t})}$, respectively. The Verilog implementation of Blink is available at GitHub repository.

Among tweakable block ciphers with comparable security levels, Blink achieves the lowest latency. Remarkably, Blink's overall latency remains nearly unchanged even when the tweak length is doubled. In contrast, QARMAv2 requires four additional rounds to maintain security under a doubled tweak length, resulting in a significant increase in latency. This efficiency stems from the underlying hash function, highlighting the capability of the THF mode to construct low-latency tweakable block ciphers. Furthermore, even when compared with conventional block ciphers, Blink's latency remains highly competitive.

9 Conclusion

This work introduces the THF mode, a framework for constructing low-latency tweakable block ciphers based on three hash functions. As a concrete instantiation, we proposed Blink, a cipher family optimized for memory encryption with a strong focus on latency. Hardware evaluations demonstrate that all Blink variants meet stringent low-latency goals, validating the practicality of the THF approach.

Looking forward, THF provides a promising direction for designing tweakable block ciphers across various low-latency scenarios. The choice and design of hash functions within this framework remain critical, directly influencing security against multi-tweak attacks. Future work may explore tailored hash function constructions to further enhance the adaptability and robustness of THF-based ciphers.

Acknowledgments

The authors acknowledge the reviewers for their insightful comments and detailed suggestions which helped improve this manuscript significantly. We are also grateful to the authors of [CGZ25] for communicating their new results to us, which helped us refine the analysis in Section 7, and we thank Roberto Avanzi for the insightful discussions regarding the hardware implementation results. This work is supported by the National Key Research and Development Program of China (2022YFB2701900), the National Cryptologic Science Foundation of China (2025NCSF01012), the National Natural Science Foundation of China (62532013), the Fellowship Funds from China Postdoctoral Science Foundation (2025M771675) and the Fundamental Research Funds for the Central Universities.

References

[ABC+24] Ravi Anand, Subhadeep Banik, Andrea Caforio, Tatsuya Ishikawa, Takanori Isobe, Fukang Liu, Kazuhiko Minematsu, Mostafizar Rahman, and Kosei Sakamoto. Gleeok: A family of low-latency prfs and its applications to authenticated encryption. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2024(2):545–587, 2024.

[ABD+23] Roberto Avanzi, S Banik, O Dunkelman, M Eichlseder, S Ghosh, M Nageler, F Regazzoni, et al. The QARMAv2 family of tweakable block ciphers. *IACR Transactions on Symmetric Cryptology*, 2023, 2023.

[ABI+18] Gianira Alfarano, Christof Beierle, Takanori Isobe, Stefan Kölbl, and Gregor Leander. Shiftrows alternatives for AES-like ciphers and optimal cell permutations for midori and skinny. *IACR Trans. Symm. Cryptol.*, 2018(2):20–47, 2018.

[AL11] Aysajan Abidin and Jan-Åke Larsson. New universal hash functions. In Frederik Armknecht and Stefan Lucks, editors, *Research in Cryptology - 4th Western European Workshop, WEWoRC 2011, Weimar, Germany, July 20-22, 2011, Revised Selected Papers*, volume 7242 of *Lecture Notes in Computer Science*, pages 99–108. Springer, 2011.

[AMD19] AMD. Secure encrypted virtualization (SEV), 2019.

[ARM21] ARM. Arm CCA Security Model, August 2021. Rev 1.0, Document Number DEN0096.

[Ava17] Roberto Avanzi. The QARMA block cipher family. *IACR Trans. Symm. Cryptol.*, 2017(1):4–44, 2017.

[BBI+15] Subhadeep Banik, Andrey Bogdanov, Takanori Isobe, Kyoji Shibutani, Harunaga Hiwatari, Toru Akishita, and Francesco Regazzoni. Midori: A block cipher for low energy. In Tetsu Iwata and Jung Hee Cheon, editors, *ASIACRYPT 2015, Part II*, volume 9453 of *LNCS*, pages 411–436. Springer, Heidelberg, November / December 2015.

[BBL+24] Augustin Bariant, Jules Baudrin, Gaëtan Leurent, Clara Pernot, Léo Perrin, and Thomas Peyrin. Fast AES-based universal hash functions and MACs featuring lemac and petitmac. *IACR Trans. Symmetric Cryptol.*, 2024(2):35–67, 2024.

[BCG+12] Julia Borghoff, Anne Canteaut, Tim Güneysu, Elif Bilge Kavun, Miroslav Knežević, Lars R. Knudsen, Gregor Leander, Ventzislav Nikov, Christof Paar, Christian Rechberger, Peter Rombouts, Søren S. Thomsen, and Tolga Yalçin. PRINCE - A low-latency block cipher for pervasive computing applications - extended abstract. In Xiaoyun Wang and Kazue Sako, editors, *ASIACRYPT 2012*, volume 7658 of *LNCS*, pages 208–225. Springer, Heidelberg, December 2012.

[BCLR17] Christof Beierle, Anne Canteaut, Gregor Leander, and Yann Rotella. Proving resistance against invariant attacks: How to choose the round constants. In Jonathan Katz and Hovav Shacham, editors, *CRYPTO 2017, Part II*, volume 10402 of *LNCS*, pages 647–678. Springer, Heidelberg, August 2017.

[BDD+23] Yanis Belkheyar, Joan Daemen, Christoph Dobraunig, Santosh Ghosh, and Shahram Rasoolzadeh. Bipbip: A low-latency tweakable block cipher with small dimensions. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pages 326–368, 2023.

[BDG+25] Yanis Belkheyar, Patrick Derbez, Shibam Ghosh, Gregor Leander, Silvia Mella, Léo Perrin, Shahram Rasoolzadeh, Lukas Stennes, Siwei Sun, Gilles Van Assche, and Damian Vizár. ChiLow and ChiChi: New constructions for code encryption. In Serge Fehr and Pierre-Alain Fouque, editors, *Advances in Cryptology – EUROCRYPT 2025*, pages 212–243, Cham, 2025. Springer Nature Switzerland.

[BEK+20] Dusan Bozilov, Maria Eichlseder, Miroslav Knezevic, Baptiste Lambin, Gregor Leander, Thorben Moos, Ventzislav Nikov, Shahram Rasoolzadeh, Yosuke Todo, and Friedrich Wiemer. PRINCEv2 - more security for (almost) no overhead. In Orr Dunkelman, Michael J. Jacobson Jr., and Colin O'Flynn, editors, *Selected Areas in Cryptography - SAC 2020 - 27th International Conference, Halifax, NS, Canada (Virtual Event), October 21-23, 2020, Revised Selected Papers*, volume 12804 of *Lecture Notes in Computer Science*, pages 483–511. Springer, 2020.

[BGGS20] Zhenzhen Bao, Chun Guo, Jian Guo, and Ling Song. TNT: How to tweak a block cipher. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part II*, volume 12106 of *LNCS*, pages 641–673. Springer, Heidelberg, May 2020.

[BGLS19] Zhenzhen Bao, Jian Guo, San Ling, and Yu Sasaki. Peigen – a platform for evaluation, implementation, and generation of S-boxes. *IACR Trans. Symm. Cryptol.*, 2019(1):330–394, 2019.

[BHK+99] John Black, Shai Halevi, Hugo Krawczyk, Ted Krovetz, and Phillip Rogaway. UMAC: fast and secure message authentication. In Michael J. Wiener, editor, *Advances in Cryptology - CRYPTO '99, 19th Annual International Cryptology Conference, Santa Barbara, California, USA, August 15-19, 1999, Proceedings*, volume 1666 of *Lecture Notes in Computer Science*, pages 216–233. Springer, 1999.

[Bie19] Armin Biere. Cadical at the sat race 2019, 2019.

[BIL+21] Subhadeep Banik, Takanori Isobe, Fukang Liu, Kazuhiko Minematsu, and Kosei Sakamoto. Orthros: A low-latency PRF. *IACR Trans. Symm. Cryptol.*, 2021(1):37–77, 2021.

[BJK+16] Christof Beierle, Jérémy Jean, Stefan Kölbl, Gregor Leander, Amir Moradi, Thomas Peyrin, Yu Sasaki, Pascal Sasdrich, and Siang Meng Sim. The SKINNY family of block ciphers and its low-latency variant MANTIS. In Matthew Robshaw and Jonathan Katz, editors, *CRYPTO 2016, Part II*, volume 9815 of *LNCS*, pages 123–153. Springer, Heidelberg, August 2016.

[BS91] Eli Biham and Adi Shamir. Differential cryptanalysis of DES-like cryptosystems. In Alfred J. Menezes and Scott A. Vanstone, editors, *CRYPTO'90*, volume 537 of *LNCS*, pages 2–21. Springer, Heidelberg, August 1991.

[CGL+23] Federico Canale, Tim Güneysu, Gregor Leander, Jan Philipp Thoma, Yosuke Todo, and Rei Ueno. {SCARF}–a {Low-Latency} block cipher for secure {Cache-Randomization}. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 1937–1954, 2023.

[CGZ25] Shiyao Chen, Jian Guo, and Tianyu Zhang. Weak tweak-key analysis of blink via superbox. Cryptology ePrint Archive, Paper 2025/2149, 2025.

[CV03] Matthew Cary and Ramarathnam Venkatesan. A message authentication code based on unimodular matrix groups. In Dan Boneh, editor, *Advances in Cryptology - CRYPTO 2003, 23rd Annual International Cryptology Conference, Santa Barbara, California, USA, August 17-21, 2003, Proceedings*, volume 2729 of *Lecture Notes in Computer Science*, pages 500–512. Springer, 2003.

[CW79] J Lawrence Carter and Mark N Wegman. Universal classes of hash functions. *Journal of Computer and System Sciences*, 18(2):143–154, 1979.

[DR02] Joan Daemen and Vincent Rijmen. *The design of Rijndael*, volume 2. Springer, 2002.

[FGGL+25] Antonio Flórez-Gutiérrez, Lorenzo Grassi, Gregor Leander, Ferdinand Sibleyras, and Yosuke Todo. General practical cryptanalysis of the sum of round-reduced block ciphers and ZIP-AES. In Kai-Min Chung and Yu Sasaki, editors, *Advances in Cryptology – ASIACRYPT 2024*, pages 280–311, Singapore, 2025. Springer Nature Singapore.

[GFAD23] Koustabh Ghosh, Jonathan Fuchs, Parisa Amiri-Eliasi, and Joan Daemen. Universal hashing based on field multiplication and (near-)MDS matrices. In Nadia El Mrabet, Luca De Feo, and Sylvain Duquesne, editors, *Progress in Cryptology - AFRICACRYPT 2023 - 14th International Conference on Cryptology in Africa, Sousse, Tunisia, July 19-21, 2023, Proceedings*, volume 14064 of *Lecture Notes in Computer Science*, pages 129–150. Springer, 2023.

[GJN+16] Jian Guo, Jérémy Jean, Ivica Nikolic, Kexin Qiao, Yu Sasaki, and Siang Meng Sim. Invariant subspace attack against Midori64 and the resistance criteria for S-box designs. *IACR Trans. Symm. Cryptol.*, 2016(1):33–56, 2016. https://tosc.iacr.org/index.php/ToSC/article/view/534.

[Gue16] Shay Gueron. Memory encryption for general-purpose processors. *IEEE Security & Privacy*, 14(6):54–62, 2016.

[HKPT24] Kai Hu, Mustafa Khairallah, Thomas Peyrin, and Quan Quan Tan. uknit: Breaking round-alignment for cipher design–featuring uknit-bc, an ultra low-latency block cipher. *Cryptology ePrint Archive*, 2024.

[HKR15] Viet Tung Hoang, Ted Krovetz, and Phillip Rogaway. Robust authenticated-encryption AEZ and the problem that it solves. In Elisabeth Oswald and Marc Fischlin, editors, *EUROCRYPT 2015, Part I*, volume 9056 of *LNCS*, pages 15–44. Springer, Heidelberg, April 2015.

[Int20] Intel. Intel trust domain extensions. Whitepaper, 2020.

[JKNS24] Ashwin Jha, Mustafa Khairallah, Mridul Nandi, and Abishanka Saha. Tight security of TNT and beyond - attacks, proofs and possibilities for the cascaded LRW paradigm. In Marc Joye and Gregor Leander, editors, *Advances in Cryptology - EUROCRYPT 2024 - 43rd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zurich, Switzerland, May 26-30, 2024, Proceedings, Part I*, volume 14651 of *Lecture Notes in Computer Science*, pages 249–279. Springer, 2024.

[JLM+19a] Ashwin Jha, Eik List, Kazuhiko Minematsu, Sweta Mishra, and Mridul Nandi. XHX – a framework for optimally secure tweakable block ciphers from classical block ciphers and universal hashing. In Tanja Lange and Orr Dunkelman, editors, *Progress in Cryptology – LATINCRYPT 2017*, pages 207–227, Cham, 2019. Springer International Publishing.

[JLM+19b] Ashwin Jha, Eik List, Kazuhiko Minematsu, Sweta Mishra, and Mridul Nandi. XHX - A framework for optimally secure tweakable block ciphers from classical block ciphers and universal hashing. In Tanja Lange and Orr Dunkelman, editors, *LATINCRYPT 2017*, volume 11368 of *LNCS*, pages 207–227. Springer, Heidelberg, September 2019.

[JN20] Ashwin Jha and Mridul Nandi. Tight security of cascaded LRW2. *Journal of Cryptology*, 33(3):1272–1317, July 2020.

[JNP14] Jérémy Jean, Ivica Nikolic, and Thomas Peyrin. Tweaks and keys for block ciphers: The TWEAKEY framework. In Palash Sarkar and Tetsu Iwata, editors, *ASIACRYPT 2014, Part II*, volume 8874 of *LNCS*, pages 274–288. Springer, Heidelberg, December 2014.

[Kra94] Hugo Krawczyk. LFSR-based hashing and authentication. In Yvo Desmedt, editor, *Advances in Cryptology - CRYPTO '94, 14th Annual International Cryptology Conference, Santa Barbara, California, USA, August 21-25, 1994, Proceedings*, volume 839 of *Lecture Notes in Computer Science*, pages 129–139. Springer, 1994.

[LAAZ11] Gregor Leander, Mohamed Ahmed Abdelraheem, Hoda AlKhzaimi, and Erik Zenner. A cryptanalysis of PRINTcipher: The invariant subspace attack. In Phillip Rogaway, editor, *CRYPTO 2011*, volume 6841 of *LNCS*, pages 206–221. Springer, Heidelberg, August 2011.

[LL18] ByeongHak Lee and Jooyoung Lee. Tweakable block ciphers secure beyond the birthday bound in the ideal cipher model. In Thomas Peyrin and Steven Galbraith, editors, *Advances in Cryptology – ASIACRYPT 2018*, pages 305–335, Cham, 2018. Springer International Publishing.

[LMMR21] Gregor Leander, Thorben Moos, Amir Moradi, and Shahram Rasoolzadeh. The SPEEDY family of block ciphers engineering an ultra low-latency cipher from gate level for secure processor architectures. *IACR TCHES*, 2021(4):510–545, 2021. https://tches.iacr.org/index.php/TCHES/article/view/9074.

[LMR15] Gregor Leander, Brice Minaud, and Sondre Rønjom. A generic approach to invariant subspace attacks: Cryptanalysis of robin, iSCREAM and Zorro. In Elisabeth Oswald and Marc Fischlin, editors, *EUROCRYPT 2015, Part I*, volume 9056 of *LNCS*, pages 254–283. Springer, Heidelberg, April 2015.

[LRW02] Moses Liskov, Ronald L. Rivest, and David Wagner. Tweakable block ciphers. In Moti Yung, editor, *CRYPTO 2002*, volume 2442 of *LNCS*, pages 31–46. Springer, Heidelberg, August 2002.

[LST12a] Will Landecker, Thomas Shrimpton, and R. Seth Terashima. Tweakable blockciphers with beyond birthday-bound security. In Reihaneh Safavi-Naini and Ran Canetti, editors, *CRYPTO 2012*, volume 7417 of *LNCS*, pages 14–30. Springer, Heidelberg, August 2012.

[LST12b] Will Landecker, Thomas Shrimpton, and R. Seth Terashima. Tweakable blockciphers with beyond birthday-bound security. In Reihaneh Safavi-Naini and Ran Canetti, editors, *Advances in Cryptology – CRYPTO 2012*, pages 14–30, Berlin, Heidelberg, 2012. Springer Berlin Heidelberg.

[MN17] Bart Mennink and Samuel Neves. Optimal PRFs from blockcipher designs. *IACR Trans. Symm. Cryptol.*, 2017(3):228–252, 2017.

[MNT90] Yishay Mansour, Noam Nisan, and Prasoon Tiwari. The computational complexity of universal hashing. In *22nd ACM STOC*, pages 235–243. ACM Press, May 1990.

[MV04] David A. McGrew and John Viega. The galois/counter mode of operation (GCM). *Submission to NIST Modes of Operation Process*, 2004.

[Nan14] Mridul Nandi. On the minimum number of multiplications necessary for universal hash functions. In Carlos Cid and Christian Rechberger, editors, *Fast Software Encryption - 21st International Workshop, FSE 2014, London, UK, March 3-5, 2014. Revised Selected Papers*, volume 8540 of *Lecture Notes in Computer Science*, pages 489–508. Springer, 2014.

[Ras22] Shahram Rasoolzadeh. Low-latency boolean functions and bijective s-boxes. *IACR Transactions on Symmetric Cryptology*, 2022(3):403–447, Sep. 2022.

[SSW23] Yaobin Shen, François-Xavier Standaert, and Lei Wang. Forgery attacks on several beyond-birthday-bound secure MACs. In Jian Guo and Ron Steinfeld, editors, *Advances in Cryptology – ASIACRYPT 2023*, pages 169–189, Singapore, 2023. Springer Nature Singapore.

[ST17] Yu Sasaki and Yosuke Todo. New impossible differential search tool from design and cryptanalysis aspects - revealing structural properties of several ciphers. In Jean-Sébastien Coron and Jesper Buus Nielsen, editors, *EUROCRYPT 2017, Part III*, volume 10212 of *LNCS*, pages 185–215. Springer, Heidelberg, April / May 2017.

[Sti92] Douglas R. Stinson. Universal hashing and authentication codes. In Joan Feigenbaum, editor, *CRYPTO'91*, volume 576 of *LNCS*, pages 74–85. Springer, Heidelberg, August 1992.

[SWW17] Ling Sun, Wei Wang, and Meiqin Wang. Automatic search of bit-based division property for ARX ciphers and word-based division property. In Tsuyoshi Takagi and Thomas Peyrin, editors, *ASIACRYPT 2017, Part I*, volume 10624 of *LNCS*, pages 128–157. Springer, Heidelberg, December 2017.

[TLS16] Yosuke Todo, Gregor Leander, and Yu Sasaki. Nonlinear invariant attack - practical attack on full SCREAM, iSCREAM, and Midori64. In Jung Hee Cheon and Tsuyoshi Takagi, editors, *ASIACRYPT 2016, Part II*, volume 10032 of *LNCS*, pages 3–33. Springer, Heidelberg, December 2016.

[Tod15] Yosuke Todo. Structural evaluation by generalized integral property. In Elisabeth Oswald and Marc Fischlin, editors, *EUROCRYPT 2015, Part I*, volume 9056 of *LNCS*, pages 287–314. Springer, Heidelberg, April 2015.

[WC81] Mark N. Wegman and Larry Carter. New hash functions and their use in authentication and set equality. *J. Comput. Syst. Sci.*, 22(3):265–279, 1981.

[WHWL24] Jianhua Wang, Tao Huang, Shuang Wu, and Zilong Liu. Twinkle: A family of low-latency schemes for authenticated encryption and pointer authentication. *IACR Communications in Cryptology*, 1(2), 2024.

A Supplementary Material for S-box Selection

The definition of extended bit permutation equivalence can be found in Definition 2. It is easy to verify that the properties of linearity, uniformity, algebraic degree, and full diffusion are the same within the same equivalence class.

Definition 2 (Extended Bit Permutation Equivalence [Ras22])**.** Two n-bit to m-bit Boolean functions f and g are called extended bit permutation equivalent if there exist a bit permutation of n bits P_i, a bit permutation of m bits P_o, $\boldsymbol{\alpha} \in \{0,1\}^n$, and $\boldsymbol{\beta} \in \{0,1\}^m$ in such a way that $g(\boldsymbol{x}) = (P_o \circ f \circ P_i(\boldsymbol{x} \oplus \boldsymbol{\alpha})) \oplus \boldsymbol{\beta}$ for all $\boldsymbol{x} \in \{0,1\}^n$.

The three representatives of the extended bit permutation equivalence classes of 4-bit S-boxes, characterized by a depth of 3.5, linearity of 8, uniformity of 4, and containing involutory S-boxes, are listed in Table 6.

Table 6: The three representatives of the equivalence classes of 4-bit S-boxes with depth 3.5, linearity 8, uniformity 4, and containing involutory S-boxes.

x	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
$S_1(x)$	0	1	2	4	3	8	f	c	9	5	b	6	7	e	d	a
$S_2(x)$	0	1	2	3	4	8	c	9	7	f	e	b	5	d	6	a
$S_3(x)$	0	1	2	3	4	c	e	8	7	d	f	b	6	5	a	9

Using the PEIGEN tool [BGLS19], their algebraic normal forms are presented below. The results show that only the first class achieves full diffusion, with each output bit exhibiting the maximum algebraic degree of 3.

B The Proof of 3-round Diffusion

Theorem 3. *Conditions 1 and 2 ensure that, in both the forward and backward directions, each output cell after three rounds is influenced by exactly 23 input cells.*

Proof. Let the input be denoted by $\boldsymbol{x_0}$, the output of round i by $\boldsymbol{x_i}$, and the input to the shuffle layer in round i by $\boldsymbol{y_i}$. Clearly, $\boldsymbol{x_{i+1}}$ is the output of the shuffle layer in round i. We prove the forward case; the backward case follows analogously.

Table 7: The algebraic normal forms of the three representatives.

$S_1(x)$	$y_0 = x_0 \oplus x_2 \oplus x_3 \oplus x_0x_1 \oplus x_0x_3 \oplus x_2x_3 \oplus x_0x_1x_2 \oplus x_0x_2x_3$ $y_1 = x_1 \oplus x_2 \oplus x_0x_1 \oplus x_0x_2 \oplus x_1x_2 \oplus x_0x_1x_2 \oplus x_0x_1x_3 \oplus x_0x_2x_3 \oplus x_1x_2x_3$ $y_2 = x_0x_1 \oplus x_0x_3 \oplus x_1x_2 \oplus x_2x_3 \oplus x_0x_1x_2 \oplus x_0x_1x_3 \oplus x_0x_2x_3 \oplus x_1x_2x_3$ $y_3 = x_3 \oplus x_0x_2 \oplus x_0x_3 \oplus x_1x_2 \oplus x_2x_3 \oplus x_0x_1x_2 \oplus x_0x_2x_3$
$S_2(x)$	$y_0 = x_0 \oplus x_3 \oplus x_0x_2 \oplus x_0x_3 \oplus x_1x_3 \oplus x_0x_1x_2 \oplus x_0x_1x_3 \oplus x_0x_2x_3$ $y_1 = x_1 \oplus x_3 \oplus x_1x_2 \oplus x_1x_3 \oplus x_2x_3$ $y_2 = x_2 \oplus x_3 \oplus x_0x_2 \oplus x_2x_3 \oplus x_0x_1x_3 \oplus x_0x_2x_3$ $y_3 = x_0x_2 \oplus x_0x_3 \oplus x_1x_2 \oplus x_1x_3 \oplus x_0x_1x_2 \oplus x_0x_1x_3 \oplus x_0x_2x_3$
$S_3(x)$	$y_0 = x_0 \oplus x_3 \oplus x_0x_2 \oplus x_0x_3 \oplus x_2x_3$ $y_1 = x_1 \oplus x_3 \oplus x_0x_3 \oplus x_1x_3 \oplus x_0x_1x_2 \oplus x_0x_1x_3$ $y_2 = x_2 \oplus x_3 \oplus x_2x_3 \oplus x_0x_1x_2 \oplus x_0x_1x_3 \oplus x_1x_2x_3$ $y_3 = x_0x_2 \oplus x_0x_3 \oplus x_1x_2 \oplus x_1x_3 \oplus x_0x_1x_2 \oplus x_0x_1x_3 \oplus x_1x_2x_3$

Due to the diffusion matrix M, each cell of $\boldsymbol{x_i}$ is influenced by 3 cells from a specific column of $\boldsymbol{x_{i-1}}$. Moreover, if two cells in $\boldsymbol{x_i}$ are influenced by cells from the same column of $\boldsymbol{x_{i-1}}$, they are collectively influenced by all 4 cells of that column.

According to Condition 1, each column of $\boldsymbol{y_2}$ is influenced by 12 distinct cells from 4 columns of $\boldsymbol{x_1}$, with each column contributing 3 cells. For convenience, we group these 3 cells from the same column into one group. Each cell within this column of $\boldsymbol{y_2}$ is then influenced by cells from three of these groups, resulting in a total of 9 distinct cells of $\boldsymbol{x_1}$. Condition 2 ensures that any three of these four groups originate from at least 7 distinct columns of $\boldsymbol{x_0}$. Upon verification, exactly two pairs of these cells come from the same column; otherwise, Condition 2 would be violated. Therefore, each cell of $\boldsymbol{y_2}$ is influenced by $5 \times 3 + 2 \times 4 = 23$ cells from $\boldsymbol{x_0}$.

Since the shuffle layer itself does not introduce additional mixing, each cell of $\boldsymbol{x_3}$ is influenced by 23 cells from $\boldsymbol{x_0}$. □

C Area comparison

Table 8 summarizes the area results of various block ciphers, which were obtained concurrently with the latency evaluation reported in Table 5.

D Round Constants

The round constants in 64-bit block version are detailed as follows.

$$\boldsymbol{rc_1} = \texttt{0x13198a2e03707344}.$$
$$\boldsymbol{rc_2} = \texttt{0x082efa98ec4e6c89}.$$
$$\boldsymbol{rc_3} = \texttt{0xbe5466cf34e90c6c}.$$
$$\boldsymbol{rc_4} = \texttt{0x3f84d5b5b5470917}.$$
$$\boldsymbol{rc_5} = \texttt{0xd1310ba698dfb5ac}.$$
$$\boldsymbol{rc'_1} = \texttt{0x0d95748f728eb658}.$$
$$\boldsymbol{rc'_2} = \texttt{0x7b54a41dc25a59b5}.$$
$$\boldsymbol{rc'_3} = \texttt{0xc5d1b023286085f0}.$$
$$\boldsymbol{rc'_4} = \texttt{0x8e79dcb0603a180e}.$$

Table 8: Area comparison of various block ciphers

Cipher	Block Size	Tweak Length	Key Length	Security Level (D, T)	Area (μm^2) Nangate 45 nm	Nangate 15 nm	ASAP 7 nm
PRINCE	64	-	128	$2^n, 2^{127-n}$	9988.83	2005.75	859.55
Midori64	64	-	128	$2^{128}, 2^{128}$	17588.45	4290.43	1599.44
Midori128	128	-	128	$2^{128}, 2^{128}$	42010.44	12385.91	3945.64
SPEEDY-6-192	192	-	192	$2^{128}, 2^{128}$	29171.16	7173.69	2520.66
MANTIS	64	64	128	$2^n, 2^{126-n}$	13685.43	2794.49	1059.34
QARMA-64-σ_0	64	64	128	$2^n, 2^{128-n}$	15381.72	3162.88	1208.10
QARMAv2$_7$-64	64	64	128	$2^{56}, 2^{128}$	16386.93	3406.09	1376.29
QARMAv2$_9$-64	64	128	128	$2^{56}, 2^{128}$	19000.91	4157.28	1651.78
QARMAv2$_9$-128	128	128	128	$2^{80}, 2^{128}$	37383.64	8668.84	3318.63
QARMAv2$_{11}$-128	128	256	128	$2^{80}, 2^{128}$	43736.78	11362.12	3886.42
QARMAv2$_{13}$-128	128	128	256	$2^{80}, 2^{256}$	50334.12	12849.91	4418.31
QARMAv2$_{15}$-128	128	256	256	$2^{80}, 2^{256}$	56114.83	15443.76	4987.45
Blink-64a	64	64	448	$2^{56}, 2^{128}$	34911.44	8782.28	2906.50
Blink-64b	64	128	448	$2^{56}, 2^{128}$	56080.25	14255.85	4544.56
Blink-128a	128	128	1024	$2^{80}, 2^{128}$	115218.43	29356.43	9319.23
Blink-128b	128	256	1024	$2^{80}, 2^{128}$	198079.82	51183.06	15686.78
Blink-128A	128	128	1280	$2^{80}, 2^{256}$	123086.18	31184.00	9958.78
Blink-128B	128	256	1280	$2^{80}, 2^{256}$	206028.70	53010.83	16387.19

$$\boldsymbol{rc'_5} = \texttt{0xd71577c1bd314b27}.$$

The round constants in 128-bit block version are detailed as follows.

$$\begin{aligned}
\boldsymbol{rc_1} &= \texttt{0x243f6a8885a308d313198a2e03707344}.\\
\boldsymbol{rc_2} &= \texttt{0xa4093822299f31d0082efa98ec4e6c89}.\\
\boldsymbol{rc_3} &= \texttt{0x452821e638d01377be5466cf34e90c6c}.\\
\boldsymbol{rc_4} &= \texttt{0xc0ac29b7c97c50dd3f84d5b5b5470917}.\\
\boldsymbol{rc_5} &= \texttt{0x9216d5d98979fb1bd1310ba698dfb5ac}.\\
\boldsymbol{rc_6} &= \texttt{0x2ffd72dbd01adfb7b8e1afed6a267e96}.\\
\boldsymbol{rc_7} &= \texttt{0xba7c9045f12c7f9924a19947b3916cf7}.\\
\boldsymbol{rc_8} &= \texttt{0x0801f2e2858efc16636920d871574e69}.\\
\boldsymbol{rc'_1} &= \texttt{0xa458fea3f4933d7e0d95748f728eb658}.\\
\boldsymbol{rc'_2} &= \texttt{0x718bcd5882154aee7b54a41dc25a59b5}.\\
\boldsymbol{rc'_3} &= \texttt{0x9c30d5392af26013c5d1b023286085f0}.\\
\boldsymbol{rc'_4} &= \texttt{0xca417918b8db38ef8e79dcb0603a180e}.\\
\boldsymbol{rc'_5} &= \texttt{0x6c9e0e8bb01e8a3ed71577c1bd314b27}.\\
\boldsymbol{rc'_6} &= \texttt{0x78af2fda55605c60e65525f3aa55ab94}.\\
\boldsymbol{rc'_7} &= \texttt{0x5748986263e8144055ca396a2aab10b6}.\\
\boldsymbol{rc'_8} &= \texttt{0xb4cc5c341141e8cea15486af7c72e993}.
\end{aligned}$$

E Supplementary Security Analysis

E.1 Differential and Linear Trails

Table 9, 10 and Table 11, 12 present the cell-level trails with the minimum number of active S-boxes for `Blink-64` and `Blink-128`, respectively. Each state in the tables represents the input difference (for differential analysis) or the input mask (for linear analysis) of the S-layer in each round.

E.2 Impossible Differential Trails

Table 13 and Table 14 present the cell-level impossible differential trails for `Blink-64` and `Blink-128`, respectively. Each impossible differential trail is characterized by its input and output differences, which correspond to the input difference of the first-round S-layer and the output difference of the last-round S-layer, respectively.

E.3 Division Trails

Table 15 and Table 16 show example division trails for `Blink-64` and `Blink-128`, respectively.

F Test Vector

F.1 `Blink-64a`

$$
\begin{aligned}
\boldsymbol{m} &= \texttt{0x0} \\
\boldsymbol{k} &= \texttt{0xd6a102d888a467e4d1d7dec33a246943e07c1dc6f302c57e762c2df9de6f0d21} \\
&\quad\ \texttt{6dd387874a0b52ce3022e0ad78c78a0697779021b38e7fa1} \\
\boldsymbol{t} &= \texttt{0x0123456789abcdef} \\
\\
\boldsymbol{c} &= \texttt{0xa4a0d10502be846e}
\end{aligned}
$$

F.2 `Blink-64b`

$$
\begin{aligned}
\boldsymbol{m} &= \texttt{0x0} \\
\boldsymbol{k} &= \texttt{0xd6a102d888a467e4d1d7dec33a246943e07c1dc6f302c57e762c2df9de6f0d21} \\
&\quad\ \texttt{6dd387874a0b52ce3022e0ad78c78a0697779021b38e7fa1} \\
\boldsymbol{t} &= \texttt{0x0123456789abcdef0123456789abcdef} \\
\\
\boldsymbol{c} &= \texttt{0x743e142f17caaae1}
\end{aligned}
$$

F.3 Blink-128a

$$
\begin{aligned}
\boldsymbol{m} &= \texttt{0x0} \\
\boldsymbol{k} &= \texttt{0xd6a102d888a467e4d1d7dec33a246943e07c1dc6f302c57e762c2df9de6f0d21} \\
&\quad \texttt{6dd387874a0b52ce3022e0ad78c78a0697779021b38e7fa15e2b66350517f80f} \\
&\quad \texttt{2961c648d578bae174d70cb769c30a45cc40300fe8a342ca57a0bd0251ae39b6} \\
&\quad \texttt{21b8f104904374bbd6a102e234a664e421b8f104904374bbd6a102d888a666e4} \\
\boldsymbol{t} &= \texttt{0x0123456789abcdef0123456789abcdef} \\
\\
\boldsymbol{c} &= \texttt{0xb722eef350bb182074a6ff13c967a593}
\end{aligned}
$$

F.4 Blink-128b

$$
\begin{aligned}
\boldsymbol{m} &= \texttt{0x0} \\
\boldsymbol{k} &= \texttt{0xd6a102d888a467e4d1d7dec33a246943e07c1dc6f302c57e762c2df9de6f0d21} \\
&\quad \texttt{6dd387874a0b52ce3022e0ad78c78a0697779021b38e7fa15e2b66350517f80f} \\
&\quad \texttt{2961c648d578bae174d70cb769c30a45cc40300fe8a342ca57a0bd0251ae39b6} \\
&\quad \texttt{21b8f104904374bbd6a102e234a664e421b8f104904374bbd6a102d888a666e4} \\
\boldsymbol{t} &= \texttt{0x0123456789abcdef0123456789abcdef0123456789abcdef0123456789abcdef} \\
\\
\boldsymbol{c} &= \texttt{0x20705a38e00412165bdabcac1dcbdec2}
\end{aligned}
$$

F.5 Blink-128A

$$
\begin{aligned}
\boldsymbol{m} &= \texttt{0x0} \\
\boldsymbol{k} &= \texttt{0xd6a102d888a467e4d1d7dec33a246943e07c1dc6f302c57e762c2df9de6f0d21} \\
&\quad \texttt{6dd387874a0b52ce3022e0ad78c78a0697779021b38e7fa15e2b66350517f80f} \\
&\quad \texttt{2961c648d578bae174d70cb769c30a45cc40300fe8a342ca57a0bd0251ae39b6} \\
&\quad \texttt{21b8f104904374bbd6a102e234a664e421b8f104904374bbd6a102d888a666e4} \\
&\quad \texttt{28962a4c96893eda752c17026a6395c2d6963be43b2fc10813d73f5a4a48d28d} \\
\boldsymbol{t} &= \texttt{0x0123456789abcdef0123456789abcdef} \\
\\
\boldsymbol{c} &= \texttt{0x82449f141c183601195b5046eac2b026}
\end{aligned}
$$

F.6 Blink-128B

$\boldsymbol{m} =$ 0x0

$\boldsymbol{k} =$ 0xd6a102d888a467e4d1d7dec33a246943e07c1dc6f302c57e762c2df9de6f0d21
6dd387874a0b52ce3022e0ad78c78a0697779021b38e7fa15e2b66350517f80f
2961c648d578bae174d70cb769c30a45cc40300fe8a342ca57a0bd0251ae39b6
21b8f104904374bbd6a102e234a664e421b8f104904374bbd6a102d888a666e4
28962a4c96893eda752c17026a6395c2d6963be43b2fc10813d73f5a4a48d28d

$\boldsymbol{t} =$ 0x0123456789abcdef0123456789abcdef0123456789abcdef0123456789abcdef

$\boldsymbol{c} =$ 0x8dc41b223bc8cd9923b1297dd27583fc

Table 9: Cell-level differential / linear trail for `Blink-64` (Half Cipher)

Half Cipher	# of active S-boxes	Difference / mask State
3 Rounds	7	1--- 1--- 1--- ---- ---- ---- -1-- ---- -1-- 1--1 ---- ----
4 Rounds	16	11-- 1--- 11-- -1-- -1-- ---- -1-- ---- ---- 1--- --1- ---- 1--- -1-- 11-- --11
5 Rounds	23	1--- ---- 1--- 1--- ---- --1- ---- ---- ---1 ---- 1--- --1- 1-11 -11- 11-1 -1-- 1-1- 11-- 1--- 1--1
6 Rounds	30	1--- 1--- ---- ---1 1-1- --1- ---- 11-- 11-- 11-- 111- ---- -1-- 11-- 111- --1- ---1 --11 ---- --11 --1- -1-- ---- --1-
7 Rounds	35	1--- 1--- ---- ---1 1-1- --1- ---- 11-- 11-- 11-- 111- ---- -1-- 11-- 111- --1- ---1 --11 ---- --11 --1- -1-- ---- --1- ---- 1--1 1-1- --1-
8 Rounds	38	1--- ---- ---- ---- ---- --1- -1-- ---1 -111 1--1 1--- 111- ---- 1-11 11-- 1111 ---- -111 -111 -111 ---- 1--- 1--- 1--- 1--- ---- ---- ---- ---- --1- -1-- ---1

Table 10: Cell-level differential / linear trail for `Blink-64` (Full Cipher)

Full Cipher	# of active S-boxes	Difference / mask State

4 Rounds	16	`1--- ---- 1--- 1---`
		`---- --1- ---- ----`
		`--1- ---- --1- --1-`
		`-111 -111 --1- -1-1`

6 Rounds	24	`--11 --11 ---- ----`
		`---- -1-- 1--- 11--`
		`-1-- ---1 -1-- ---1`
		`-1-- ---1 -1-- ---1`
		`---- -1-- 1--- 11--`
		`--11 --11 ---- ----`

8 Rounds	32	`1--- ---- 1--- 1---`
		`---- --1- ---- ----`
		`---1 ---- 1--- --1-`
		`1-11 -11- 11-1 -1--`
		`1-11 -11- 11-1 -1--`
		`---1 ---- 1--- --1-`
		`---- --1- ---- ----`
		`1--- ---- 1--- 1---`

10 Rounds	50	`1--- ---- 1--- 1---`
		`---- --1- ---- ----`
		`---1 ---- 1--- --1-`
		`1-11 -11- 11-1 -1--`
		`1-11 11-- 1--- 1-11`
		`-111 1--- 11-- -111`
		`-11- -1-1 1-11 1-1-`
		`-1-- ---1 --1- ----`
		`-1-- ---- ---- ----`
		`-1-- ---- -1-- -1--`

12 Rounds	64	`1-1- 111- -1-- ---1`
		`111- -11- 1-1- 11--`
		`---- --1- --1- --1-`
		`---- ---- 1--- ----`
		`1--- --1- -1-- ----`
		`-1-1 1-11 11-- --11`
		`-1-1 1-11 11-- --11`
		`1--- --1- -1-- ----`
		`---- ---- 1--- ----`
		`---- --1- --1- --1-`
		`111- -11- 1-1- 11--`
		`1-1- 111- -1-- ---1`

Table 11: Cell-level differential / linear trail for `Blink-128` (Half Cipher)

Half Cipher	# of active S-boxes	Difference / mask State

3 Rounds	7	`1------- 1------- -------- 1-------`
		`-------1 -------- -------- --------`
		`-------- -------- -------- -1-1-1--`

4 Rounds	16	`------1- -1----1- -1------ -1----1-`
		`---1---- ---1---- -------- --------`
		`-------- ----1--- ---1---- --------`
		`--1----- 1---11-- ---1---1 --------`

5 Rounds	24	`-------- -1----11 -1----11 ------11`
		`----11-- -------- -----1-- ----1---`
		`1-1----- 1-1----- -------- --------`
		`------1- ------1- 1------- 1-------`
		`-------1 -1------ -----1-- --1-----`

6 Rounds	36	`-------- -1----11 -1----11 ------11`
		`----11-- -------- -----1-- ----1---`
		`1-1----- 1-1----- -------- --------`
		`------1- ------1- 1------- 1-------`
		`-------1 -1------ -----1-- --1-----`
		`1--11-1- -------1 111----- -1-1-11-`

7 Rounds	48	`1---11-- 1-1----- 1---1--- --1-----`
		`--1---1- --1----1 -------1 --1---11`
		`-------- -------- --1-11-- ----1---`
		`1-----1- 1------1 1------1 ------11`
		`-------- -------- ----11-- --1-1---`
		`1-----1- 1------1 1-1----1 ------1-`
		`------1- -------- 1---11-- -11--1-1`

8 Rounds	58	`--11---- --1--1-- --11---- -----1--`
		`-------- -------1 ---1--1- ------11`
		`-------- ---111-- ---11--- ---1-1--`
		`-1------ -------1 ---1---1 ------1-`
		`----11-- -1-111-- -1-1-1-- -1-111--`
		`---1---- -------1 ---1---1 ------1-`
		`-------- -1-1---- ---1-11- -1---1--`
		`-----1-- -11-1--- -1--111- ------1-`

Table 12: Cell-level differential / linear trail for `Blink-128` (Full Cipher)

Full Cipher	# of active S-boxes	Difference / mask State

4 Rounds	16	`-------- -----1-- 1-1--1-- 1-1-----`
		`-------1 --1----1 --1----- --1----1`
		`--1----- -------- -------1 --------`
		`----1--- -------- ----1--- --------`

6 Rounds	24	`----11-- -------- -----1-- ----1---`
		`1-1----- 1-1----- -------- --------`
		`------1- ------1- 1------- 1-------`
		`------1- ------1- 1------- 1-------`
		`1-1----- 1-1----- -------- --------`
		`----11-- -------- -----1-- ----1---`

8 Rounds	40	`-------- -1----11 -1----11 ------11`
		`----11-- -------- -----1-- ----1---`
		`1-1----- 1-1----- -------- --------`
		`------1- ------1- 1------- 1-------`
		`------1- ------1- 1------- 1-------`
		`1-1----- 1-1----- -------- --------`
		`----11-- -------- -----1-- ----1---`
		`-------- -1----11 -1----11 ------11`

10 Rounds	64	`-1----1- -1------ -1----1- ------1-`
		`-------- -1------ -1------ --------`
		`----11-- -------- -------- --------`
		`-1------ 1-1----1 -------1 ------1-`
		`----11-1 -1-1---- 111--1-- 111111-1`
		`----11-1 -1-1---- 111--1-- 111111-1`
		`-1------ 1-1----1 -------1 ------1-`
		`----11-- -------- -------- --------`
		`-------- -1------ -1------ --------`
		`-1----1- -1------ -1----1- ------1-`

Table 13: Cell-level impossible differential trail for `Blink-64` (Full Cipher)

Full Cipher	Difference
$6+0$ Rounds	`1--- ---- ---- ----` `1--- ---- ---- ----`
$5+2$ Rounds	`1--- ---- ---- ----` `1--- ---- ---- ----`
$3+3$ Rounds	`1--- ---- ---- ----` `1--- ---- ---- ----`

Table 14: Cell-level impossible differential trail for `Blink-128` (Full Cipher)

Full Cipher	Difference
$9+0$ Rounds	`1------- -------- -------- --------` `-------- ------1- -------- --------`
$8+1$ Rounds	`1------- -------- -------- --------` `-------- 1------- -------- --------`
$7+2$ Rounds	`1------- -------- -------- --------` `--1----- -------- -------- --------`
$6+3$ Rounds	`1------- -------- -------- --------` `----1--- -------- -------- --------`
$5+4$ Rounds	`1------- -------- -------- --------` `-------- ----1--- -------- --------`

Table 15: Division trail for `Blink-64` (Half Cipher)

Round Index	Operation	State
Input		`7fff ffff ffff ffff`
Round 1	`S`	`4fff ffff ffff ffff`
	`M`	`dfff 7fff ffff efff`
	`P`	`dfff ff7f feff ffff`
Round 2	`S`	`9fff ff4f f2ff ffff`
	`M`	`deef ffdf b77f fbff`
	`P`	`dff7 edfb ef7f fffb`
Round 3	`S`	`1ff1 21f2 2f4f fff2`
	`M`	`6dd7 0b7a b7e0 3ff3`
	`P`	`6b0e d70f d373 7afb`
Round 4	`S`	`8c08 110f 7121 4cfc`
	`M`	`706b ddbd 0d0d 0000`
	`P`	`7dd0 0bd0 60d0 bd00`
Round 5	`S`	`4810 0220 2040 8400`
	`M`	`0000 e000 0e00 0070`
	`P`	`0000 00e0 00e0 0070`
Round 6	`S`	`0000 0020 0080 0040`
	`M`	`00e0 0000 0000 0000`
	`P`	`0000 0000 e000 0000`
Round 7	`S`	`0000 0000 8000 0000`
	`M`	`8000 0000 0000 0000`
	`P`	`8000 0000 0000 0000`
Output		`8000 0000 0000 0000`

Table 16: Division trail for `Blink-128` (Half Cipher)

Round Index	Operation	State			
Input		7fffffff	ffffffff	ffffffff	ffffffff
Round 1	S	1fffffff	ffffffff	ffffffff	ffffffff
	M	dfffffff	ffffffff	bfffffff	7fffffff
	P	fffffffb	ffffffff	ffffffff	df7fffff
Round 2	S	fffffff2	ffffffff	ffffffff	df4fffff
	M	dfeffff7	ff7ffffe	ffffffff	ffdfffb
	P	ffffff7f	ffffffff	efffffff	dffb7efd
Round 3	S	ffffff9f	ffffffff	9fffffff	7ff8a2f4
	M	fffbf7de	dffdefb7	7ffebbff	bffffefd
	P	7efffff7	fbbfdfdf	fffbfdeb	ffbde7ef
Round 4	S	48fffff4	fb8f8f2f	fff2f49c	ffb482ef
	M	eedfd0ec	fbeedfff	ddb7e70d	7bb0eeb6
	P	0ddedbed	ef70e0ff	dbbfbe7e	ed76cfeb
Round 5	S	01484821	8f1090ff	482f2218	28441f82
	M	eb4c9008	09007bb3	0c000e0d	007f50f3
	P	079bc900	5be00f0b	400cf000	ed038307
Round 6	S	04422100	48800f08	8004f001	21011804
	M	e000bb0d	00070000	0dc00d00	00007000
	P	b0b0d000	70d07000	00c00000	e000d000
Round 7	S	80202000	40104000	00400000	20008000
	M	00000000	00000000	e000e000	00700000
	P	0000000e	00000000	0000000e	00000007
Round 8	S	00000008	00000000	00000002	00000001
	M	00000000	0000000b	00000000	00000000
	P	00000000	00000000	00000000	00000b00
Round 9	S	00000000	00000000	00000000	00000800
	M	00000800	00000000	00000000	00000000
	P	80000000	00000000	00000000	00000000
Output		80000000	00000000	00000000	00000000